Microservices Solution Docs

From Monolith to Event-Driven Microservices — Definitions, Hands-On Labs, Real-World Usage & Architecture Diagrams

Tech Stack: Java · Spring Boot · Kafka · PostgreSQL · Cassandra · Redis/Caffeine · Docker · Kubernetes · CI/CD (Canary, Blue-Green)

July 11, 2026

Contents

1 Introduction **5**

2 Part I: Architectural Foundations **6**
- 2.1 Monolithic Architecture . . . 6
  - 2.1.1 Definition . . . 6
  - 2.1.2 Hands-On: Runnable Mini-Project . . . 6
  - 2.1.3 Real-Time Usage . . . 8
  - 2.1.4 Architectural Flow Diagram . . . 8
- 2.2 Modular Monolith Architecture . . . 9
  - 2.2.1 Definition . . . 9
  - 2.2.2 Hands-On: Runnable Mini-Project . . . 9
  - 2.2.3 Real-Time Usage . . . 11
  - 2.2.4 Architectural Flow Diagram . . . 11
- 2.3 Microservices, Event-Driven Microservices . . . 12
  - 2.3.1 Definition . . . 12
  - 2.3.2 Hands-On: Runnable Mini-Project . . . 12
  - 2.3.3 Real-Time Usage . . . 14
  - 2.3.4 Architectural Flow Diagram . . . 14
- 2.4 Serverless Architectures — Designed to Handle Millions of Requests . . . 16
  - 2.4.1 Definition . . . 16
  - 2.4.2 Hands-On: Runnable Mini-Project . . . 16
  - 2.4.3 Real-Time Usage . . . 18
  - 2.4.4 Architectural Flow Diagram . . . 18

3 Part II: Building Microservices the Right Way **19**
- 3.1 Microservices Decomposition . . . 19
  - 3.1.1 Definition . . . 19
  - 3.1.2 Hands-On: Runnable Mini-Project . . . 19
  - 3.1.3 Real-Time Usage . . . 20
  - 3.1.4 Architectural Flow Diagram . . . 20
- 3.2 Microservices Communications (Sync / Async Message-Based Communication, API Gateways) . . . 21
  - 3.2.1 Definition . . . 21
  - 3.2.2 Hands-On: Runnable Mini-Project . . . 21
  - 3.2.3 Real-Time Usage . . . 23
  - 3.2.4 Architectural Flow Diagram . . . 23

3.3 Microservices Data Management (Databases / Query / Commands / Distributed Transactions) . . . 25
3.3.1 Definition . . . 25
3.3.2 Hands-On: Runnable Mini-Project . . . 25
3.3.3 Real-Time Usage . . . 28
3.3.4 Architectural Flow Diagram . . . 28
3.4 Microservices Distributed Caching . . . 29
3.4.1 Definition . . . 29
3.4.2 Hands-On: Runnable Mini-Project . . . 29
3.4.3 Real-Time Usage . . . 30
3.4.4 Architectural Flow Diagram . . . 30
3.5 Microservices Deployments with Containers and Orchestrators (Applying Microservices Design Patterns) . . . 31
3.5.1 Definition . . . 31
3.5.2 Hands-On: Runnable Mini-Project . . . 31
3.5.3 Real-Time Usage . . . 32
3.5.4 Architectural Flow Diagram . . . 32

4 Part III: Architecture Deep-Dives & Cross-Cutting Patterns 34
4.1 3-Tier Architecture . . . 34
4.1.1 Definition . . . 34
4.1.2 Hands-On: Runnable Mini-Project . . . 34
4.1.3 Real-Time Usage . . . 35
4.1.4 Architectural Flow Diagram . . . 35
4.2 Microservices Architectures (Reference Architecture Deep-Dive) . . . 36
4.2.1 Definition . . . 36
4.2.2 Hands-On: Runnable Mini-Project . . . 36
4.2.3 Real-Time Usage . . . 37
4.2.4 Architectural Flow Diagram . . . 37
4.3 Microservices Architectures — API Gateways (Deep-Dive) . . . 39
4.3.1 Definition . . . 39
4.3.2 Hands-On: Runnable Mini-Project . . . 39
4.3.3 Real-Time Usage . . . 40
4.3.4 Architectural Flow Diagram . . . 40
4.4 Microservices Architectures — API Gateway (BFF: Backend For Frontend) . . . 42
4.4.1 Definition . . . 42
4.4.2 Hands-On: Runnable Mini-Project . . . 42
4.4.3 Real-Time Usage . . . 42
4.4.4 Architectural Flow Diagram . . . 43
4.5 Microservices Architecture — Service Aggregator / Registry Patterns . . . 46
4.5.1 Definition . . . 46
4.5.2 Hands-On: Runnable Mini-Project . . . 46
4.5.3 Real-Time Usage . . . 47
4.5.4 Architectural Flow Diagram . . . 47
4.6 Microservices Architecture — Message Broker . . . 50
4.6.1 Definition . . . 50
4.6.2 Hands-On: Runnable Mini-Project . . . 50
4.6.3 Real-Time Usage . . . 51
4.6.4 Architectural Flow Diagram . . . 51
4.7 Microservices Architecture — Database Sharding (Cassandra) . . . 53
4.7.1 Definition . . . 53
4.7.2 Hands-On: Runnable Mini-Project . . . 53
4.7.3 Real-Time Usage . . . 54
4.7.4 Architectural Flow Diagram . . . 54

4.8 Event-Driven Microservices Architecture (Deep-Dive: Event Sourcing & Streaming) 56
4.8.1 Definition . . . 56
4.8.2 Hands-On: Runnable Mini-Project . . . 56
4.8.3 Real-Time Usage . . . 58
4.8.4 Architectural Flow Diagram . . . 58
4.9 Microservices Distributed Caching (Caffeine, Redis, Cache Eviction: LRU, LFU, TTL) 59
4.9.1 Definition . . . 59
4.9.2 Hands-On: Runnable Mini-Project . . . 59
4.9.3 Real-Time Usage . . . 60
4.9.4 Architectural Flow Diagram . . . 60
4.10 Microservices Containers and Orchestrators with CI/CD (Canary, Blue-Green, Logs, Metrics) . . . 62
4.10.1 Definition . . . 62
4.10.2 Hands-On: Runnable Mini-Project . . . 62
4.10.3 Real-Time Usage . . . 64
4.10.4 Architectural Flow Diagram . . . 65
4.11 Microservices Resilience4j — Retry, Rate Limiting (Algorithms), Bulkhead . . . 66
4.11.1 Definition . . . 66
4.11.2 Hands-On: Runnable Mini-Project . . . 66
4.11.3 Real-Time Usage . . . 68
4.11.4 Architectural Flow Diagram . . . 68

5 Part IV: Migration Playbook — Monolith to Microservices 70
5.1 Phase 1: Decouple the Monolith . . . 70
5.1.1 Definition . . . 70
5.1.2 Hands-On: Runnable Mini-Project . . . 70
5.1.3 Real-Time Usage . . . 72
5.1.4 Architectural Flow Diagram . . . 72
5.2 Phase 2: Develop Local Microservices . . . 73
5.2.1 Definition . . . 73
5.2.2 Hands-On: Runnable Mini-Project . . . 73
5.2.3 Real-Time Usage . . . 75
5.2.4 Architectural Flow Diagram . . . 75
5.3 Phase 3: Implement Local Microservices . . . 76
5.3.1 Definition . . . 76
5.3.2 Hands-On: Runnable Mini-Project . . . 76
5.3.3 Real-Time Usage . . . 77
5.3.4 Architectural Flow Diagram . . . 78
5.4 Phase 4: Deploy Microservices to the Cloud . . . 79
5.4.1 Definition . . . 79
5.4.2 Hands-On: Runnable Mini-Project . . . 79
5.4.3 Real-Time Usage . . . 80
5.4.4 Architectural Flow Diagram . . . 80
5.5 Phase 5: Implement Microservices in the Cloud . . . 81
5.5.1 Definition . . . 81
5.5.2 Hands-On: Runnable Mini-Project . . . 81
5.5.3 Real-Time Usage . . . 82
5.5.4 Architectural Flow Diagram . . . 82
5.6 Phase 6: Decommission the Monolith . . . 83
5.6.1 Definition . . . 83
5.6.2 Hands-On: Runnable Mini-Project . . . 83
5.6.3 Real-Time Usage . . . 84
5.6.4 Architectural Flow Diagram . . . 84
5.7 Migration Playbook — Summary . . . 84

6 Conclusion **85**

1 Introduction

This document is a step-by-step, hands-on solution guide that evolves a system's architecture from a single **Monolith**, through a **Modular Monolith**, into full **Microservices** (including **Event-Driven Microservices** and **Serverless**), and then goes deep on the cross-cutting concerns every microservices platform needs in production: decomposition, communication, data management, caching, deployment, resilience, and CI/CD (Canary & Blue-Green releases) with logs and metrics.

Every topic follows the same structure:

1. **Definition** — what it is and when to use it
2. **Hands-On** — a runnable Spring Boot mini-project (Java 17+, Maven, Docker Compose)
3. **Real-Time Usage** — how this is used in production systems at scale
4. **Architectural Flow Diagram** — a diagram of the request/data flow

Baseline tech stack used throughout: Java 17, Spring Boot 3.x, Spring Cloud, Apache Kafka, PostgreSQL, Apache Cassandra, Redis / Caffeine, Resilience4j, Docker, Kubernetes, Jenkins/GitHub Actions, Prometheus + Grafana, ELK/Loki.

The chapters that follow trace exactly this journey:

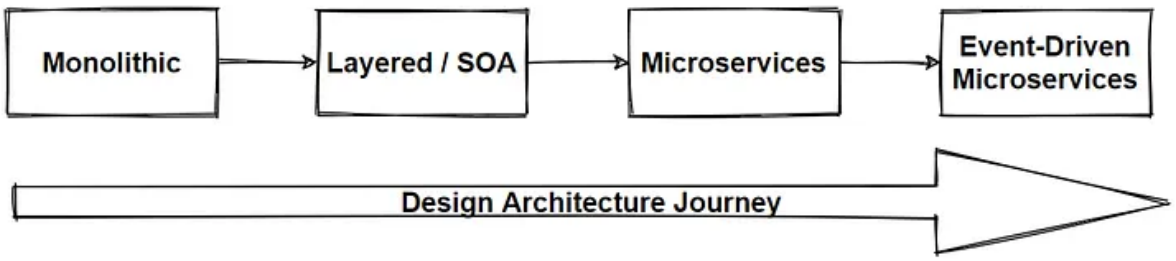


Figure 1: Design Architecture Journey

2 Part I: Architectural Foundations

We start with a single running example — an **E-Commerce Order Platform** (Catalog, Cart, Orders, Payments, Inventory, Notifications) — and evolve it across every architecture style so you can see the *same business problem* solved differently at each stage.

2.1 Monolithic Architecture

2.1.1 Definition

A **Monolith** packages the entire application — UI/API layer, business logic, and data access — as a **single deployable unit** (one WAR/JAR) running against a **single shared database**. All modules (Catalog, Cart, Orders, Payments) live in one codebase, one build pipeline, and scale together as one process.

2.1.2 Hands-On: Runnable Mini-Project

Project layout

```
ecommerce-monolith/
 ├─ pom.xml
 └─ src/main/java/com/acme/shop/
      ├─ ShopMonolithApplication.java
      ├─ catalog/  (Product entity, ProductController)
      ├─ cart/     (Cart entity, CartController)
      ├─ order/    (Order entity, OrderController, OrderService)
      ├─ payment/  (PaymentService - calls a fake gateway)
      └─ common/   (JPA config)
```

`pom.xml` (key dependencies)

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
  </dependency>
</dependencies>
```

`ShopMonolithApplication.java`

```java
@SpringBootApplication
public class ShopMonolithApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShopMonolithApplication.class, args);
    }
}
```

order/Order.java

```
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long productId;
    private Integer quantity;
    private BigDecimal totalAmount;
    @Enumerated(EnumType.STRING)
    private OrderStatus status;
    // getters/setters omitted for brevity
}
```

order/OrderService.java — every step happens in-process, one transaction

```
@Service
public class OrderService {

    private final OrderRepository orderRepo;
    private final ProductRepository productRepo;   // same JVM, same DB
    private final PaymentService paymentService;    // direct method call, no
     ↪  network

    public OrderService(OrderRepository orderRepo, ProductRepository
     ↪  productRepo,
                        PaymentService paymentService) {
        this.orderRepo = orderRepo;
        this.productRepo = productRepo;
        this.paymentService = paymentService;
    }

    @Transactional
    public Order placeOrder(Long productId, int qty) {
        Product product = productRepo.findById(productId)
                .orElseThrow(() -> new NoSuchElementException("Product not
                 ↪  found"));

        if (product.getStock() < qty) throw new IllegalStateException("Out of
         ↪  stock");

        product.setStock(product.getStock() - qty);
        productRepo.save(product);

        Order order = new Order();
        order.setProductId(productId);
        order.setQuantity(qty);
        order.setTotalAmount(product.getPrice().multiply(BigDecimal.valueOf(
↪  qty)));
        order.setStatus(OrderStatus.CREATED);
        orderRepo.save(order);

        paymentService.charge(order.getTotalAmount());   // in-process call
        order.setStatus(OrderStatus.PAID);
```

```
        return orderRepo.save(order);
    }
}
```

docker-compose.yml

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]
  shop-monolith:
    build: .
    ports: ["8080:8080"]
    depends_on: [postgres]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/shop
```

Run: `docker compose up --build` → `POST http://localhost:8080/orders {"productId":1,"quantity":2}`

2.1.3 Real-Time Usage

- Early-stage startups and MVPs (fastest way to ship a first version).
- Internal enterprise tools with low traffic and a small team.
- Systems with strong transactional consistency needs across all modules (one local DB transaction covers everything).
- Classic examples in their early years: early **Shopify**, early **Etsy**, early **Amazon** (before their 2001-2002 service decomposition).

2.1.4 Architectural Flow Diagram

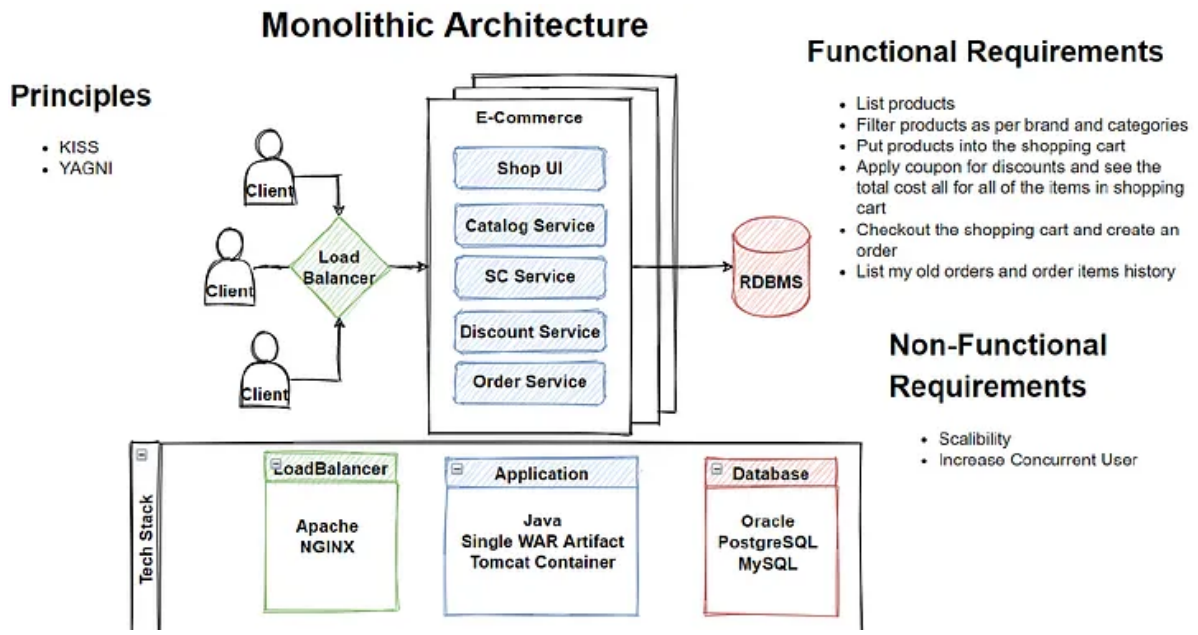


Figure 2: Monolithic Architecture

2.2 Modular Monolith Architecture

2.2.1 Definition

A **Modular Monolith** is still a single deployable unit, but the codebase is split into strongly-bounded **modules** (packages/Maven modules) with **explicit public interfaces** and **no cross-module database table access**. Each module owns its own tables/schema and communicates with other modules only through well-defined Java interfaces or in-process domain events (e.g., Spring's `ApplicationEventPublisher`). This is the natural stepping stone before microservices — it forces clean bounded contexts while you still enjoy a single deployment and single transaction store.

2.2.2 Hands-On: Runnable Mini-Project

Multi-module Maven layout

```
ecommerce-modular-monolith/
  ├─ pom.xml                (parent, packaging=pom)
  ├─ catalog-module/        (own schema: catalog.*)
  ├─ order-module/          (own schema: orders.*)
  ├─ payment-module/        (own schema: payments.*)
  └─ app/                   (assembles all modules into one Spring Boot jar)
```

Public contract exposed by `catalog-module` (other modules depend only on this interface, never on Catalog's JPA entities)

```java
public interface CatalogFacade {
    ProductView getProduct(Long productId);
    void reserveStock(Long productId, int qty);
}

public record ProductView(Long id, String name, BigDecimal price, int stock) {}
```

`order-module` depends only on the facade — not on Catalog's internal `Product` entity

```
@Service
public class OrderService {
    private final CatalogFacade catalog;          // interface, not concrete
     ↪  entity
    private final ApplicationEventPublisher events; // in-process domain events
    private final OrderRepository orderRepo;

    public OrderService(CatalogFacade catalog, ApplicationEventPublisher
     ↪  events,
                        OrderRepository orderRepo) {
        this.catalog = catalog;
        this.events = events;
        this.orderRepo = orderRepo;
    }

    @Transactional
    public Order placeOrder(Long productId, int qty) {
        ProductView product = catalog.getProduct(productId);
        if (product.stock() < qty) throw new IllegalStateException("Out of
         ↪  stock");

        catalog.reserveStock(productId, qty);

        Order order = new Order(productId, qty,
 ↪  product.price().multiply(BigDecimal.valueOf(qty)));
        orderRepo.save(order);

        events.publishEvent(new OrderPlacedEvent(order.getId(),
 ↪  order.getTotalAmount()));
        return order;
    }
}
```

`payment-module` listens to the in-process event — no direct coupling to `OrderService`

```
@Component
public class PaymentEventListener {
    private final PaymentService paymentService;

    public PaymentEventListener(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    @EventListener
    @Async
    public void onOrderPlaced(OrderPlacedEvent event) {
        paymentService.charge(event.orderId(), event.amount());
    }
}
```

Enforcing module boundaries at build time with ArchUnit

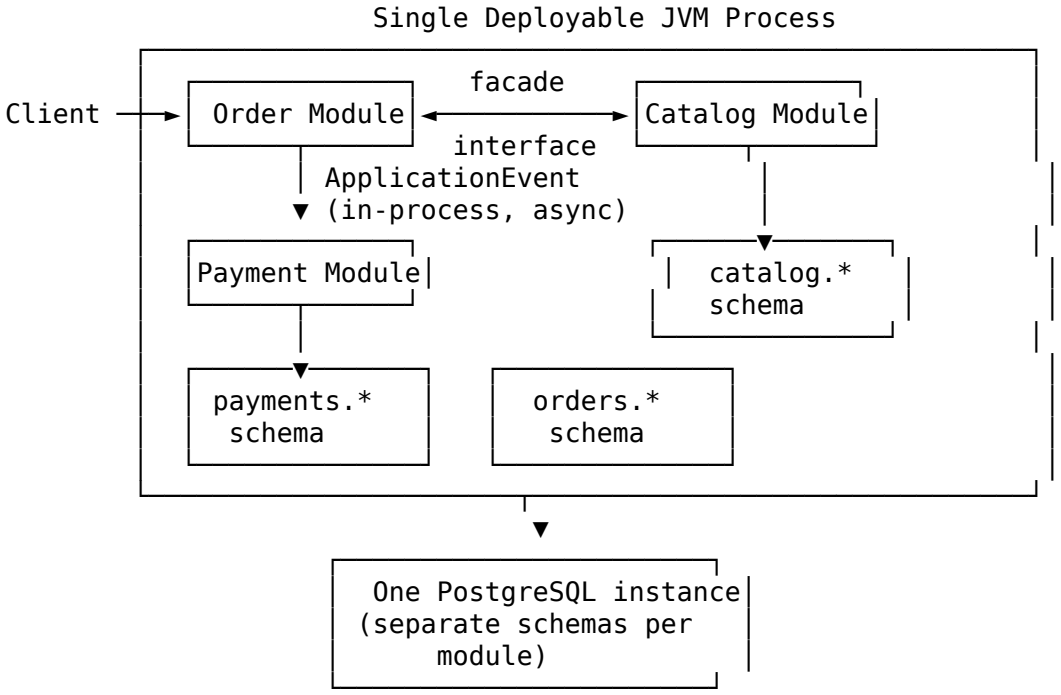
```java
@AnalyzeClasses(packages = "com.acme.shop")
class ModuleBoundaryTest {

    @ArchTest
    static final ArchRule modules_must_not_reach_into_others_internals =
        noClasses().that().resideInAPackage("..order..")
            .should().dependOnClassesThat().resideInAPackage(
             ↪  "..catalog.internal..");
}
```

2.2.3 Real-Time Usage

- **Shopify** famously runs a modular monolith (“majestic monolith”) at massive scale rather than microservices.
- Mid-sized SaaS companies that need clean boundaries and independent team ownership but don’t yet have the operational maturity (or need) for distributed systems.
- A deliberate **strangler-fig staging step**: teams extract one module at a time into a real microservice once it proves it needs independent scaling/deployment.

2.2.4 Architectural Flow Diagram



2.3 Microservices, Event-Driven Microservices

2.3.1 Definition

Microservices split the system into small, independently deployable services, each owning its own database, built and scaled independently, communicating over the network (REST/gRPC for sync, messaging for async). **Event-Driven Microservices** go further: instead of services calling each other directly, they **publish domain events** to a broker (Kafka), and interested services **subscribe and react** — decoupling producers from consumers in time and topology.

2.3.2 Hands-On: Runnable Mini-Project

Three independent Spring Boot services + Kafka, each with its **own** database.

`order-service` — publishes an event after persisting the order (own PostgreSQL DB)

```
@Service
public class OrderService {
    private final OrderRepository orderRepo;
    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;

    public OrderService(OrderRepository orderRepo,
                        KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate) {
        this.orderRepo = orderRepo;
        this.kafkaTemplate = kafkaTemplate;
    }

    @Transactional
    public Order placeOrder(OrderRequest req) {
        Order order = orderRepo.save(new Order(req.productId(), req.qty(),
↪   OrderStatus.CREATED));

        OrderPlacedEvent event = new OrderPlacedEvent(order.getId(),
↪   req.productId(),
                                                      req.qty(), Instant.now());
        // Outbox pattern in real systems (see Data Management chapter);
        // simplified direct publish shown here:
        kafkaTemplate.send("order.placed", order.getId().toString(), event);
        return order;
    }
}
```

`application.yml` (order-service)

```
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:postgresql://order-db:5432/orders
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      value-serializer:
↪   org.springframework.kafka.support.serializer.JsonSerializer
```

`inventory-service` — consumes the event, reserves stock, emits its own event

```java
@Component
public class OrderPlacedConsumer {
    private final InventoryService inventoryService;
    private final KafkaTemplate<String, StockReservedEvent> kafkaTemplate;

    public OrderPlacedConsumer(InventoryService inventoryService,
                               KafkaTemplate<String, StockReservedEvent>
 ↪  kafkaTemplate) {
        this.inventoryService = inventoryService;
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "order.placed", groupId = "inventory-service")
    public void handle(OrderPlacedEvent event) {
        boolean reserved = inventoryService.reserve(event.productId(),
         ↪  event.qty());
        var outcome = new StockReservedEvent(event.orderId(), reserved);
        kafkaTemplate.send("stock.reserved", event.orderId().toString(),
 ↪  outcome);
    }
}
```

`notification-service` — consumes final outcome, sends the customer an email/SMS

```java
@KafkaListener(topics = "stock.reserved", groupId = "notification-service")
public void onStockReserved(StockReservedEvent event) {
    String message = event.reserved()
        ? "Your order #" + event.orderId() + " is confirmed!"
        : "Sorry, order #" + event.orderId() + " is out of stock.";
    notifier.send(message);
}
```

`docker-compose.yml` for the whole event-driven cluster

```yaml
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment: { ZOOKEEPER_CLIENT_PORT: 2181 }
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on: [zookeeper]
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  order-db: { image: postgres:16, environment: { POSTGRES_DB: orders } }
  inventory-db: { image: postgres:16, environment: { POSTGRES_DB: inventory } }
  order-service: { build: ./order-service, depends_on: [kafka, order-db],
 ↪  ports: ["8081:8080"] }
  inventory-service: { build: ./inventory-service, depends_on: [kafka,
 ↪  inventory-db], ports: ["8082:8080"] }
  notification-service: { build: ./notification-service, depends_on: [kafka],
 ↪  ports: ["8083:8080"] }
```

2.3.3 Real-Time Usage

- **Uber**, **Netflix**, **Amazon**: hundreds/thousands of independently deployed services.
- Event-driven pattern powers order-fulfillment pipelines (e-commerce), ride-matching (Uber), fraud-detection pipelines (banks) where each stage reacts to an event stream instead of being called synchronously.
- Enables independent scaling: `inventory-service` can scale to 50 pods during a flash sale while `notification-service` stays at 2.

2.3.4 Architectural Flow Diagram

Plain microservices — each client talks directly to each service, each service owns its DB:

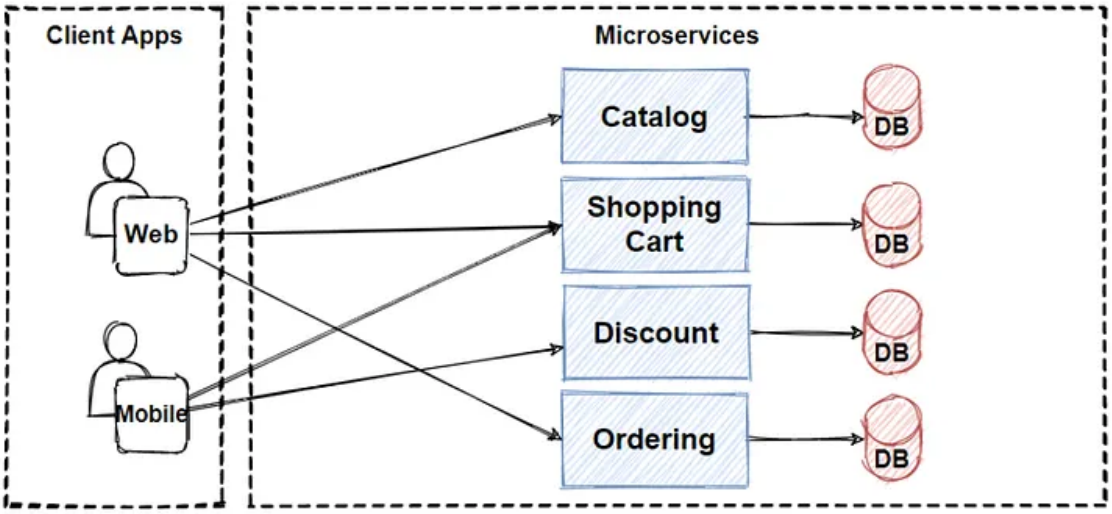


Figure 3: Microservices Architecture (basic)

Event-driven variant — services publish/consume through a shared event backbone instead of calling each other directly:

With a gateway in front and a message broker behind — the typical shape once a plain microservices system grows up:

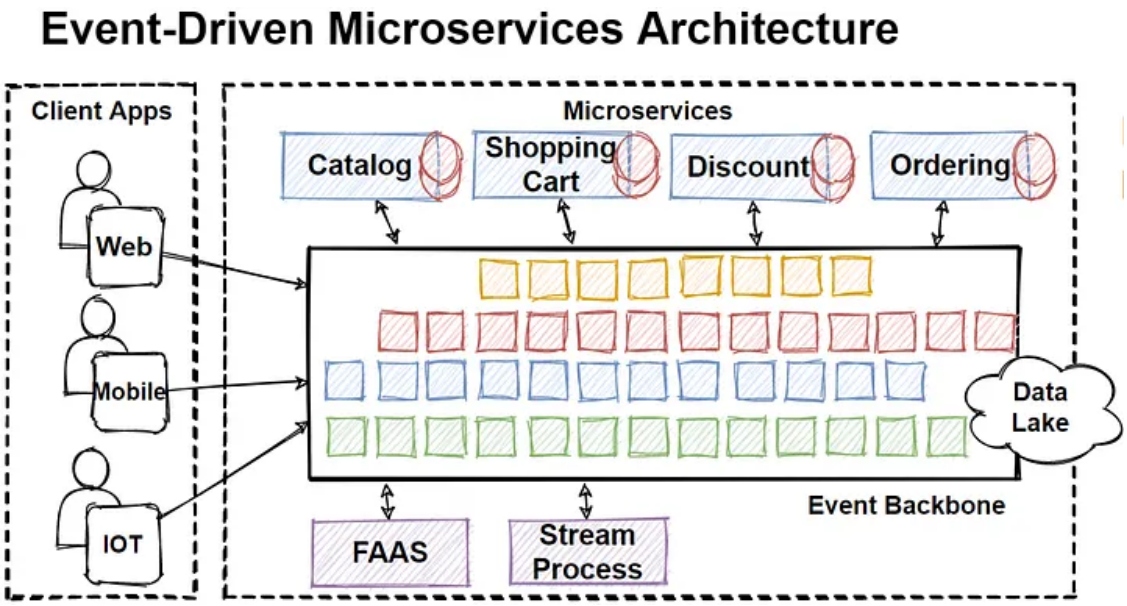


Figure 4: Event-Driven Microservices Architecture

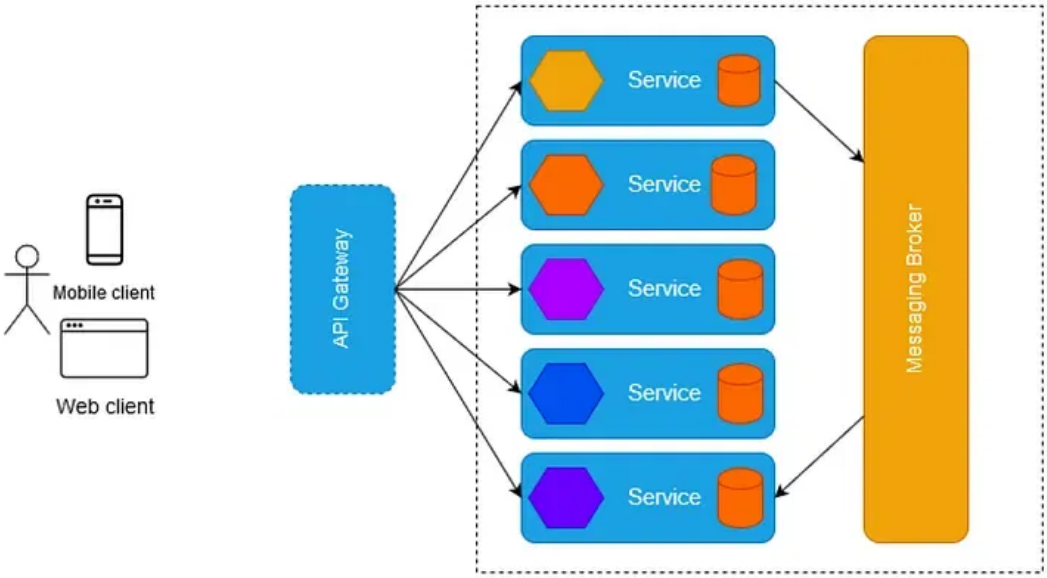


Figure 5: API Gateway → Services (each with its own DB) → Messaging Broker

2.4 Serverless Architectures — Designed to Handle Millions of Requests

2.4.1 Definition

Serverless means you deploy individual **functions** (or fully-managed containers) and the cloud provider handles provisioning, scaling (including to zero), and patching. You pay per invocation/duration, not per idle server. For millions of requests, the design combines: API Gateway → Function → Async queue/stream for fan-out → managed, auto-scaling data stores (DynamoDB/Aurora Serverless) → CDN/edge caching to shave off repeat traffic before it ever reaches a function.

This is exactly the kind of non-functional target serverless is designed to absorb — note how expected latency becomes the open question as concurrency climbs:

Concurrent Users	Requests/second	Latency (Expected)
2K	0.5K	
20K	12K	
100K	80K	<= 2 sec
500K	300K	?

Figure 6: Concurrent Users vs Requests/Second vs Latency

2.4.2 Hands-On: Runnable Mini-Project

Using **Spring Cloud Function** (portable across AWS Lambda / Azure Functions / Knative) fronted by an API Gateway, with SQS as a buffering layer to absorb spikes.

`pom.xml`

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-function-adapter-aws</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-function-web</artifactId>
</dependency>
```

`OrderFunction.java` — the entire "service" is a pure function

```
@Component
public class OrderFunction implements Function<OrderRequest, OrderResponse> {

    private final SqsClient sqsClient; // buffer for the slow/expensive part

    public OrderFunction(SqsClient sqsClient) { this.sqsClient = sqsClient; }

    @Override
    public OrderResponse apply(OrderRequest request) {
        String orderId = UUID.randomUUID().toString();
        // Do only the fast, cheap validation synchronously...
        if (request.qty() <= 0) throw new IllegalArgumentException("Invalid
        ↪  qty");
```

```
        // ...push the heavy lifting (stock check, payment) onto a queue
        sqsClient.sendMessage(SendMessageRequest.builder()
                .queueUrl(System.getenv("ORDER_QUEUE_URL"))
                .messageBody(toJson(new OrderPlacedMessage(orderId, request)))
                .build());

        return new OrderResponse(orderId, "ACCEPTED");
    }
}
```

`template.yaml` (AWS SAM) — API Gateway → Lambda → SQS → downstream Lambda, autoscaling built-in

```
Resources:
  OrderFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler:
↪   org.springframework.cloud.function.adapter.aws.FunctionInvoker::handleRequest
      Runtime: java17
      MemorySize: 512
      ReservedConcurrentExecutions: 0   # unset = scale to provider account
↪   limit
      Events:
        Api:
          Type: Api
          Properties: { Path: /orders, Method: post }
      Environment:
        Variables:
          ORDER_QUEUE_URL: !Ref OrderQueue

  OrderQueue:
    Type: AWS::SQS::Queue
    Properties:
      VisibilityTimeout: 60

  OrderProcessorFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler:
↪   org.springframework.cloud.function.adapter.aws.FunctionInvoker::handleRequest
      Runtime: java17
      Events:
        SQSEvent:
          Type: SQS
          Properties: { Queue: !GetAtt OrderQueue.Arn, BatchSize: 10 }
```

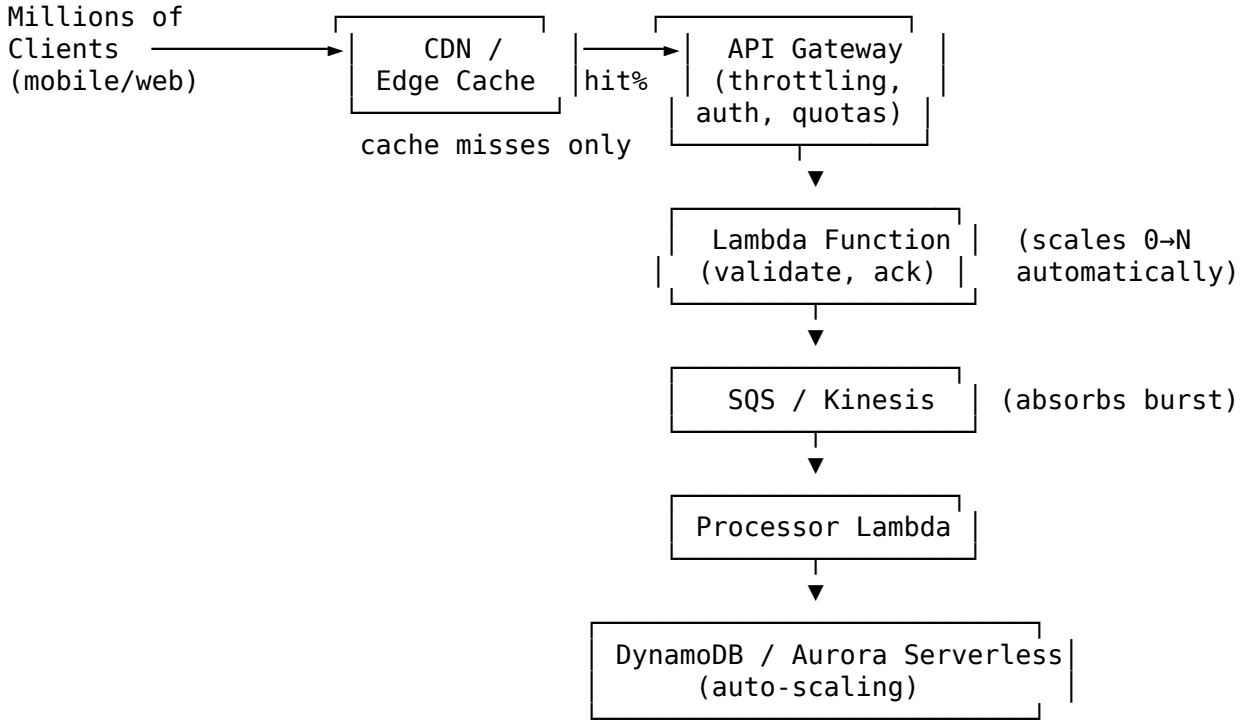
Guarding against cold starts & thundering herds

```
# Provisioned concurrency keeps N instances warm for predictable p99 latency
AutoPublishAlias: live
ProvisionedConcurrencyConfig:
  ProvisionedConcurrentExecutions: 20
```

2.4.3 Real-Time Usage

- **iRobot, Netflix (parts of the encoding pipeline), Coca-Cola vending telemetry**, and most **Black-Friday flash-sale front doors** — API Gateway + Lambda absorbs an unpredictable spike, then a queue smooths the burst before it hits a database.
- Image/video thumbnail generation, webhook receivers, chatbot backends: spiky, event-shaped traffic that would otherwise mean over-provisioning idle servers.
- CDN (CloudFront/Fastly) in front of Lambda@Edge/Workers is a standard pattern to serve millions of read requests with near-zero function invocations.

2.4.4 Architectural Flow Diagram



3 Part II: Building Microservices the Right Way

3.1 Microservices Decomposition

3.1.1 Definition

Decomposition is the discipline of deciding *where the service boundaries go*. The two dominant techniques are **Decompose by Business Capability** (what the business does: Catalog, Ordering, Payments) and **Decompose by Subdomain** using **Domain-Driven Design (DDD)** — identifying **Bounded Contexts** via **Context Mapping**, separating **Core**, **Supporting**, and **Generic** subdomains. Good decomposition targets high cohesion inside a service and loose coupling between services (each service should be independently deployable without lock-step releases).

3.1.2 Hands-On: Runnable Mini-Project

Event Storming → Bounded Contexts (worked example for the e-commerce platform)

```
Domain Events discovered:  ProductViewed, CartItemAdded, OrderPlaced,
                           PaymentAuthorized, StockReserved, OrderShipped

Bounded Contexts derived:
  Catalog Context   → owns Product, Category, Pricing
  Cart Context      → owns Cart, CartItem
  Ordering Context  → owns Order, OrderLine   (CORE subdomain — differentiator)
  Payment Context   → owns Payment, Ledger     (integrates 3rd-party gateway)
  Inventory Context → owns Stock, Warehouse
  Shipping Context  → owns Shipment, Carrier
```

Applying the rule "a service should own its data exclusively" — repository per service, never shared

```java
// order-service — repository only ever touches the orders schema
public interface OrderRepository extends JpaRepository<Order, Long> {}

// order-service NEVER does this (shared DB anti-pattern):
// @Query("SELECT p FROM Product p WHERE p.id = :id")  // NOT: Product belongs
 ↪  to Catalog!
```

Anti-corruption layer when integrating with a legacy/external system

```java
@Component
public class LegacyErpAntiCorruptionLayer {
    private final LegacyErpClient legacyClient;

    public ProductView toProductView(String legacySkuXml) {
        LegacySkuDto dto = legacyClient.parse(legacySkuXml);
        // Translate legacy vocabulary ("SKU_CD", "UNIT_PRC") into our clean
         ↪  domain model
        return new ProductView(dto.getSkuCd(), dto.getDesc(), new
         ↪  BigDecimal(dto.getUnitPrc()));
    }
}
```

Strangler Fig migration script — routing an increasing % of traffic from monolith to new service

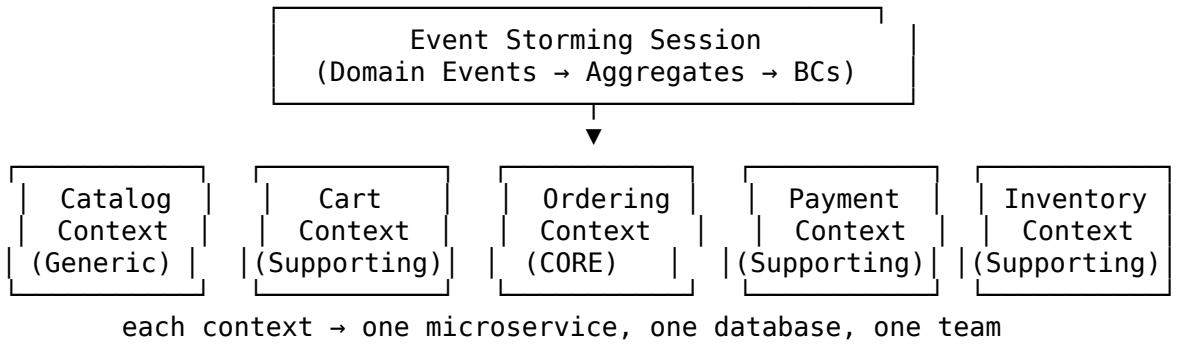
```java
@RestController
public class CatalogStranglerController {
    @Value("${strangler.catalog.new-service-percent:0}")
    private int newServicePercent;

    @GetMapping("/products/{id}")
    public ProductView getProduct(@PathVariable Long id) {
        if (ThreadLocalRandom.current().nextInt(100) < newServicePercent) {
           return newCatalogServiceClient.getProduct(id);   // new microservice
        }
        return legacyMonolithCatalogService.getProduct(id);  // old monolith
    }
}
```

3.1.3 Real-Time Usage

- **Netflix's** original 2009 decomposition split by business capability (streaming, billing, recommendations).
- Banks decomposing core banking monoliths use DDD context mapping to isolate **Ledger** (core, high consistency) from **Notifications** (generic, can be eventually consistent).
- The **Strangler Fig pattern** is the standard playbook (Martin Fowler) for migrating a legacy monolith to microservices without a risky big-bang rewrite.

3.1.4 Architectural Flow Diagram



3.2 Microservices Communications (Sync / Async Message-Based Communication, API Gateways)

3.2.1 Definition

Services talk to each other in two fundamental modes: **Synchronous** (request/response — REST, gRPC — caller blocks and waits, tight temporal coupling) and **Asynchronous message-based** (caller publishes to a broker and moves on — Kafka/RabbitMQ — loose temporal coupling, better resilience to downstream slowness). An **API Gateway** sits at the edge to give external clients one entry point that handles routing, auth, rate-limiting, and request aggregation, hiding the internal topology.

The general shape of async, message-based communication is the **Publish-Subscribe** pattern — one publisher, many independent subscribers, fully decoupled through an event bus:

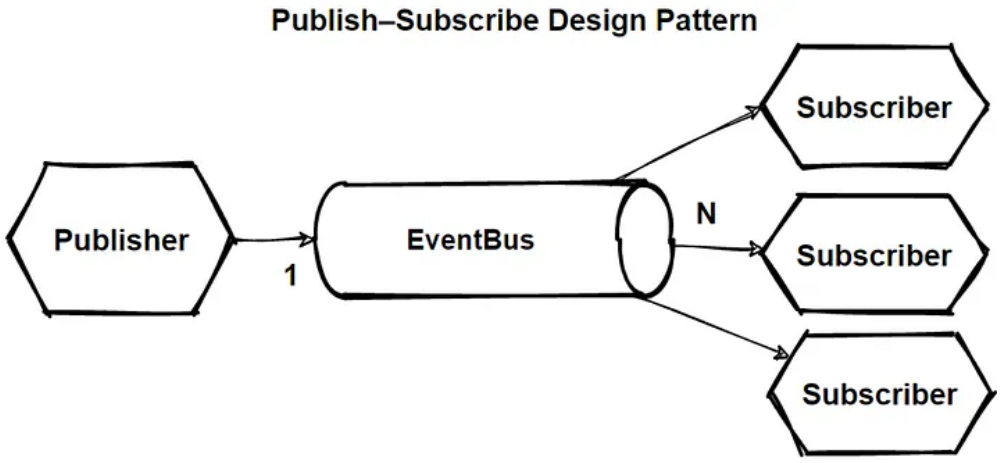


Figure 7: Publish-Subscribe Design Pattern

3.2.2 Hands-On: Runnable Mini-Project

Synchronous call with Spring Cloud OpenFeign + Resilience4j fallback

```
@FeignClient(name = "inventory-service", fallback =
 ↪  InventoryClientFallback.class)
public interface InventoryClient {
    @GetMapping("/inventory/{productId}")
    StockResponse checkStock(@PathVariable Long productId);
}

@Component
class InventoryClientFallback implements InventoryClient {
    public StockResponse checkStock(Long productId) {
        return new StockResponse(productId, false); // fail-safe: assume
         ↪  unavailable
    }
}
```

gRPC alternative (lower latency, strongly-typed, binary protocol) — `inventory.proto`

```
service InventoryService {
  rpc CheckStock (StockRequest) returns (StockResponse);
}
message StockRequest  { int64 product_id = 1; }
message StockResponse { int64 product_id = 1; bool available = 2; }
```

```
@GrpcClient("inventory-service")
private InventoryServiceGrpc.InventoryServiceBlockingStub inventoryStub;

public boolean isAvailable(long productId) {
    return inventoryStub.checkStock(StockRequest.newBuilder()
            .setProductId(productId).build()).getAvailable();
}
```

Async message-based with Kafka (fire-and-forget, retry-friendly)

```
@Service
public class ShippingRequestPublisher {
    private final KafkaTemplate<String, ShipmentRequested> kafka;
    public void requestShipment(Long orderId, Address address) {
        kafka.send("shipment.requested", orderId.toString(),
                   new ShipmentRequested(orderId, address, Instant.now()));
    }
}
```

API Gateway with Spring Cloud Gateway — routing, rate limiting, auth

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates: [ "Path=/api/orders/**" ]
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100
                redis-rate-limiter.burstCapacity: 200
            - name: CircuitBreaker
              args: { name: orderCB, fallbackUri: "forward:/fallback/orders" }
        - id: catalog-service
          uri: lb://catalog-service
          predicates: [ "Path=/api/catalog/**" ]
```

Gateway-level JWT auth filter

```
@Component
public class JwtAuthGatewayFilter implements GlobalFilter {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
     ↪  chain) {
        String token =
         ↪  exchange.getRequest().getHeaders().getFirst("Authorization");
        if (token == null || !jwtValidator.isValid(token)) {
```

```
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
        return chain.filter(exchange);
    }
}
```

3.2.3 Real-Time Usage

- Netflix's edge gateway (**Zuul**, later evolving toward Spring Cloud Gateway) fronts thousands of internal services for tens of millions of devices.
- Payment authorization is typically **synchronous** (customer waits for approval) while receipt emailing, loyalty-points updates, and analytics are **asynchronous**.
- gRPC is favored for internal service-to-service calls with tight latency budgets (e.g., ride-matching in Uber, ad-bidding in ad-tech, sub-10ms SLAs).

3.2.4 Architectural Flow Diagram

Synchronous chained calls — a client request fans out into a chain of blocking service-to-service calls:

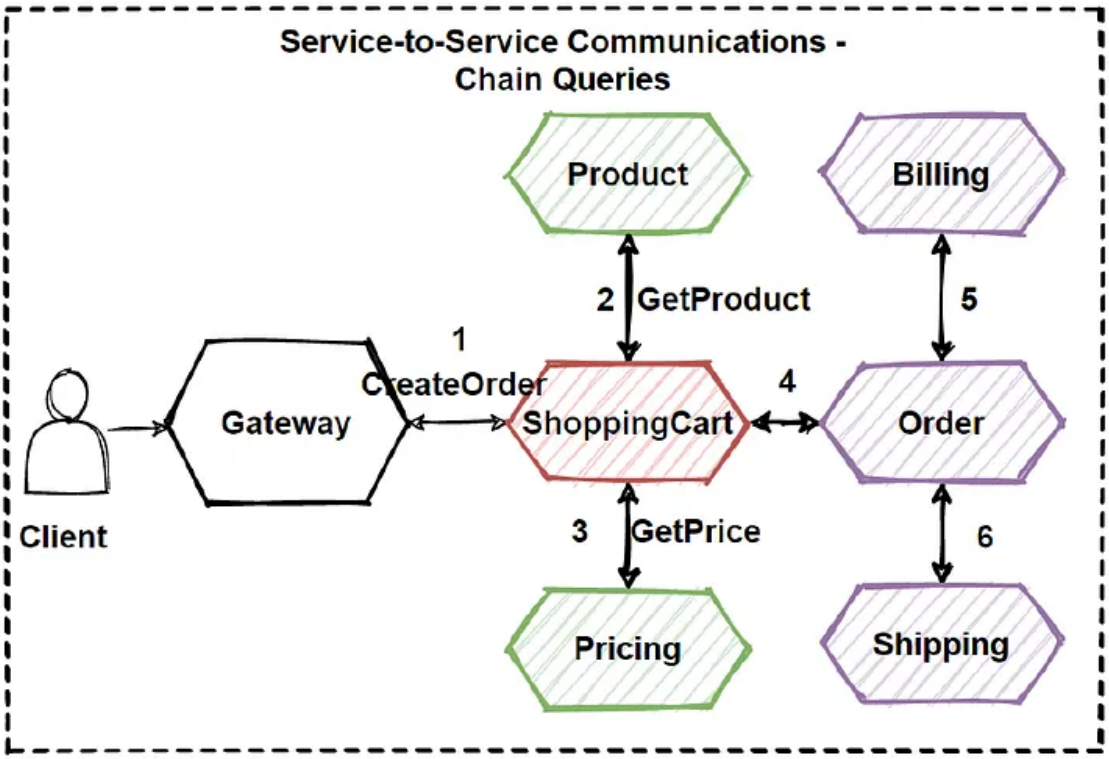


Figure 8: Service-to-Service Communications — Chain Queries

Asynchronous alternative — the same interactions decoupled through queues instead of direct blocking calls:

Combining both in one picture — the API Gateway handles synchronous client-facing traffic while every service also publishes to a shared broker for cross-service async events:

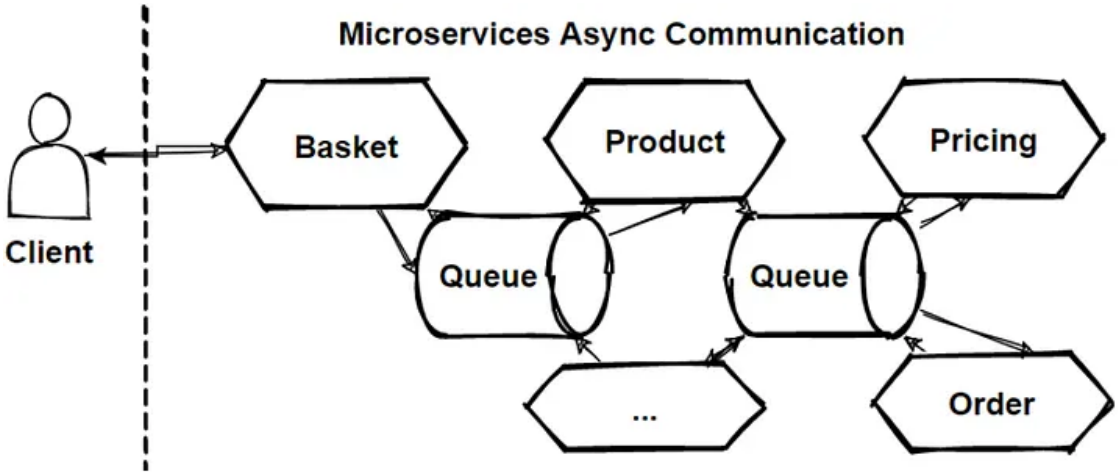


Figure 9: Microservices Async Communication

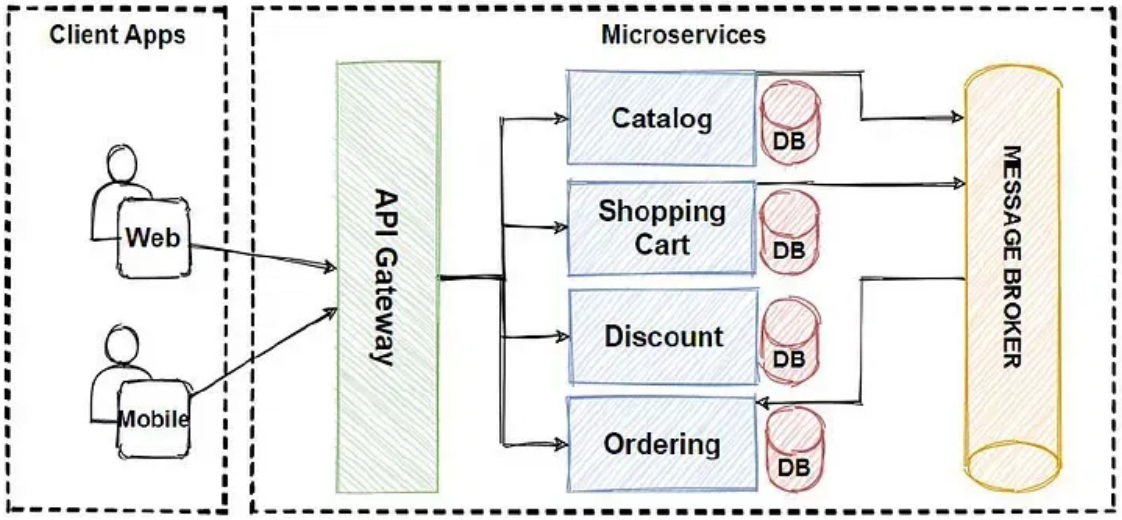


Figure 10: API Gateway + Microservices + Message Broker

3.3 Microservices Data Management (Databases / Query / Commands / Distributed Transactions)

3.3.1 Definition

Each microservice owns a **private database** (polyglot persistence: PostgreSQL for Orders, Cassandra for high-write event logs, Redis for sessions). Because a business transaction now spans multiple services/databases, you cannot use a single local ACID transaction. Two complementary patterns solve this:

- **CQRS (Command Query Responsibility Segregation)** — separate the write model (commands, normalized, strongly consistent) from the read model (queries, denormalized views built from events, fast/scalable reads).
- **Saga Pattern** — a distributed transaction implemented as a sequence of local transactions, each publishing an event that triggers the next step, with **compensating transactions** to undo prior steps on failure (Choreography via events, or Orchestration via a central saga coordinator).

The **Transactional Outbox** pattern guarantees a DB write and an event publish happen atomically (no dual-write problem).

CQRS visualized — writes and reads take separate paths, connected only by eventual consistency:

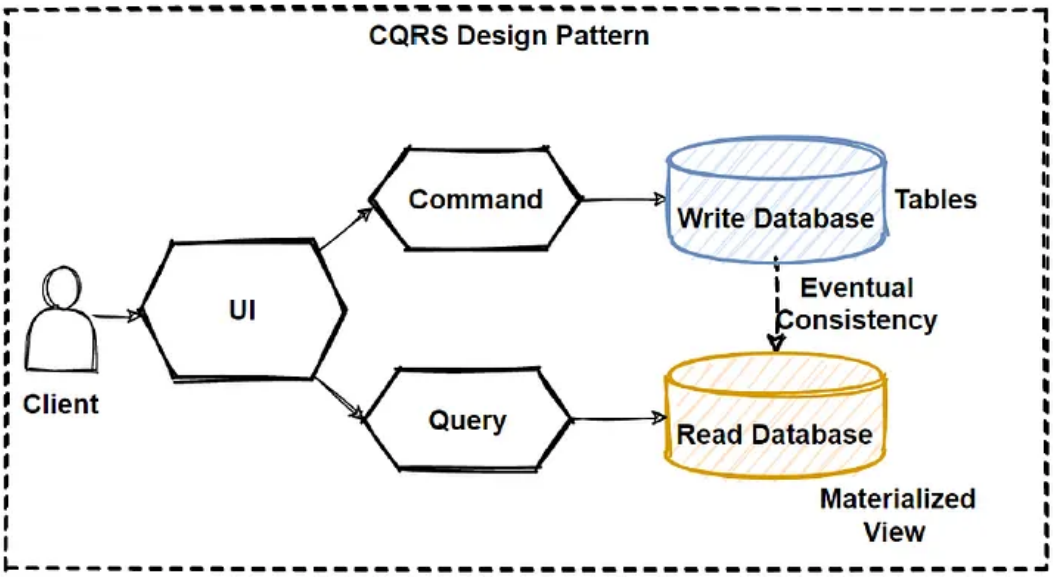


Figure 11: CQRS Design Pattern

Putting it all together at the platform level — CQRS, Event Sourcing, and eventual consistency across a full microservices topology:

3.3.2 Hands-On: Runnable Mini-Project

Transactional Outbox (Order Service, PostgreSQL) — write the order and the event in the same local transaction

```
@Transactional
public Order placeOrder(OrderRequest req) {
    Order order = orderRepo.save(new Order(req.productId(), req.qty(),
↪   OrderStatus.CREATED));
```

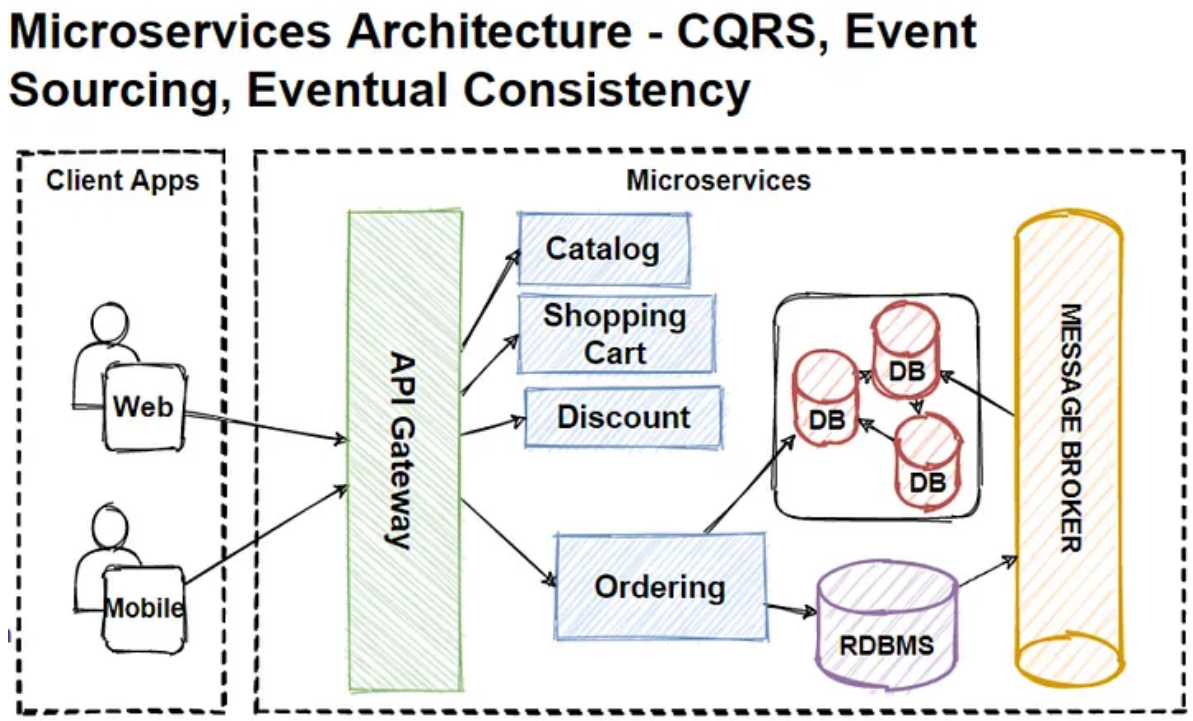


Figure 12: Microservices Architecture — CQRS, Event Sourcing, Eventual Consistency

```
    OutboxEvent evt = new OutboxEvent("Order", order.getId().toString(),
                                      "OrderPlaced", toJson(order));
    outboxRepo.save(evt);        // same local transaction as the order insert
 ↪  — atomic!
    return order;
}
```

```
// Debezium (CDC) or a polling publisher reads the outbox table and reliably
 ↪  pushes to Kafka
@Scheduled(fixedDelay = 500)
public void publishPendingEvents() {
    outboxRepo.findUnpublished().forEach(evt -> {
        kafkaTemplate.send(evt.getEventType(), evt.getPayload());
        outboxRepo.markPublished(evt.getId());
    });
}
```

Saga — Choreography style (each service reacts to the previous event and emits the next, or a compensating event)

```
// order-service
@KafkaListener(topics = "payment.failed")
public void onPaymentFailed(PaymentFailedEvent e) {
    orderService.cancelOrder(e.orderId());          // compensating transaction
    kafkaTemplate.send("order.cancelled", e.orderId());
}
```

```
// inventory-service listens for order.cancelled and releases the reserved
 ↪  stock
@KafkaListener(topics = "order.cancelled")
public void onOrderCancelled(OrderCancelledEvent e) {
    inventoryService.releaseReservation(e.orderId());  // compensating
 ↪  transaction
}
```

Saga — Orchestration style (a central coordinator drives every step explicitly)

```
@Component
public class OrderSagaOrchestrator {

    public void handle(OrderPlacedEvent e) {
        try {
            inventoryClient.reserve(e.productId(), e.qty());
            paymentClient.charge(e.orderId(), e.amount());
            shippingClient.schedule(e.orderId());
            orderRepo.updateStatus(e.orderId(), OrderStatus.CONFIRMED);
        } catch (InventoryUnavailableException ex) {
            orderRepo.updateStatus(e.orderId(), OrderStatus.CANCELLED);
        } catch (PaymentFailedException ex) {
           inventoryClient.release(e.productId(), e.qty());      // compensate
            orderRepo.updateStatus(e.orderId(), OrderStatus.CANCELLED);
        }
    }
}
```

CQRS — write model (PostgreSQL, normalized) and read model (materialized view, built from events)

```
// Write side: normal JPA entity, strongly consistent
@Entity class Order { Long id; Long productId; int qty; OrderStatus status; }

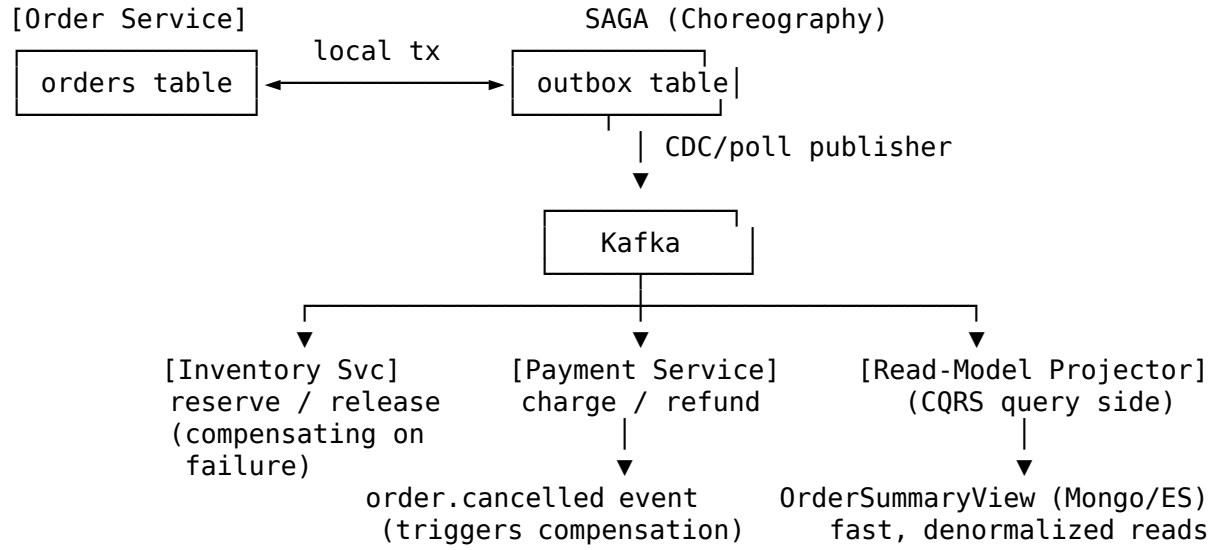
// Read side: denormalized projection updated asynchronously by an event
 ↪  listener
@Document(collection = "order_summary_view")
public class OrderSummaryView {
    String orderId, customerName, productName, status;
    BigDecimal totalAmount;
}

@KafkaListener(topics = {"order.placed", "order.shipped"})
public void projectOrderEvent(OrderEvent event) {
    OrderSummaryView view = viewRepo.findById(event.orderId())
            .orElseGet(OrderSummaryView::new);
    view.apply(event);            // update denormalized fields
    viewRepo.save(view);           // read model - MongoDB/Elasticsearch,
 ↪  optimized for queries
}
```

3.3.3 Real-Time Usage

- **Amazon's** order pipeline uses Saga-like compensation: if payment fails after inventory is reserved, a compensating "release stock" event fires.
- **Uber's** trip-pricing and driver-matching read models are CQRS projections built from an event stream, kept eventually consistent with the write-side trip ledger.
- Transactional Outbox + Debezium CDC is the de-facto standard to avoid the classic "dual write" bug (DB commit succeeds, Kafka publish fails, systems go out of sync).

3.3.4 Architectural Flow Diagram



3.4 Microservices Distributed Caching

3.4.1 Definition

Distributed Caching puts a shared, fast, in-memory data store (Redis/Memcached) between services and their databases so that repeated reads of hot data don't hit the DB every time. It's "distributed" because multiple service instances share the *same* cache cluster (unlike a local in-JVM cache), which keeps data consistent across all replicas of a service.

3.4.2 Hands-On: Runnable Mini-Project

Spring Boot + Redis, cache-aside pattern

```
@Service
public class ProductService {

    private final ProductRepository repo;
    private final RedisTemplate<String, ProductView> redis;

    @Cacheable(value = "products", key = "#productId")
    public ProductView getProduct(Long productId) {
        // Only executes on a cache MISS — Spring transparently checks Redis
        ↪ first
        return repo.findById(productId)
                .map(this::toView)
                .orElseThrow();
    }

    @CacheEvict(value = "products", key = "#productId")
    public void updatePrice(Long productId, BigDecimal newPrice) {
        repo.updatePrice(productId, newPrice);   // invalidate stale cache
↪   entry on write
    }
}
```

application.yml

```
spring:
  cache:
    type: redis
  data:
    redis:
      host: redis
      port: 6379
      timeout: 2000ms
cache:
  ttl-seconds: 300
```

```
@Bean
public RedisCacheConfiguration cacheConfiguration() {
    return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofSeconds(300))
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                    .fromSerializer(new GenericJackson2JsonRedisSerializer()));
}
```

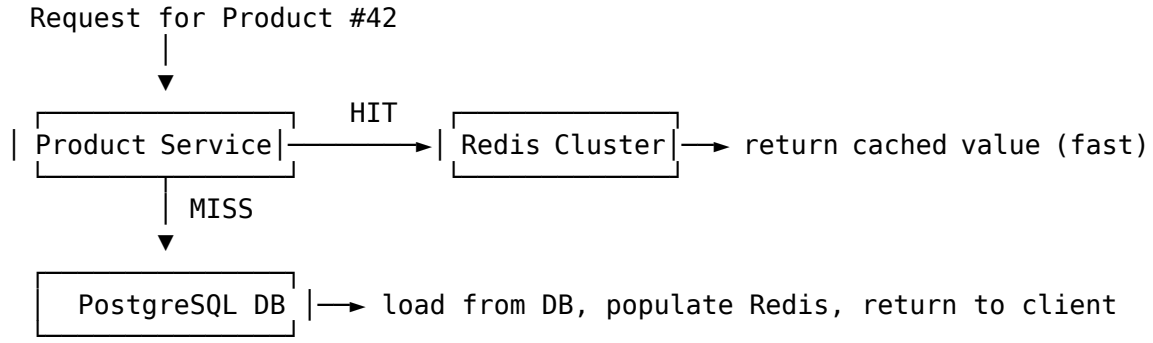
docker-compose.yml

```
services:
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    command: ["redis-server", "--maxmemory", "256mb", "--maxmemory-policy",
↪   "allkeys-lru"]
```

3.4.3 Real-Time Usage

- Product catalog pages (millions of reads, few writes) are the textbook cache-aside candidate — **Amazon**, **Flipkart** product-detail pages are cached aggressively.
- Session storage in a horizontally-scaled fleet: Redis holds session state so any pod can serve any user (no sticky sessions needed).
- Rate-limiter counters (see API Gateway section) are stored in Redis so all gateway instances share the same counter.

3.4.4 Architectural Flow Diagram



3.5 Microservices Deployments with Containers and Orchestrators (Applying Microservices Design Patterns)

3.5.1 Definition

Each microservice is packaged as a **Docker container image** (consistent runtime everywhere) and deployed onto **Kubernetes**, which handles scheduling, self-healing, scaling, service discovery, and rolling updates. On top of the platform, standard **microservices deployment patterns** are applied: **Sidecar** (attach a helper container — e.g., a service mesh proxy — to every pod), **Ambassador** (proxy outbound calls), **Service Registry & Discovery**, and **Single Service per Host/Container**.

3.5.2 Hands-On: Runnable Mini-Project

Dockerfile (multi-stage build for a small final image)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q -B package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/order-service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Kubernetes Deployment + Service + HPA (auto-scaling by CPU)

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service }
spec:
  replicas: 3
  selector: { matchLabels: { app: order-service } }
  template:
    metadata: { labels: { app: order-service } }
    spec:
      containers:
        - name: order-service
          image: registry.acme.com/order-service:1.4.2
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "500m", memory: "768Mi" }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
        - name: envoy-sidecar          # Sidecar pattern: service mesh proxy
          image: envoyproxy/envoy:v1.30-latest
          ports: [{ containerPort: 15001 }]
---
apiVersion: v1
```

```yaml
kind: Service
metadata: { name: order-service }
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8080 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-service-hpa }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: order-service
 ↪  }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization:
 ↪  70 } }
```

Service discovery — Kubernetes DNS resolves `order-service` from any pod, no manual registry needed

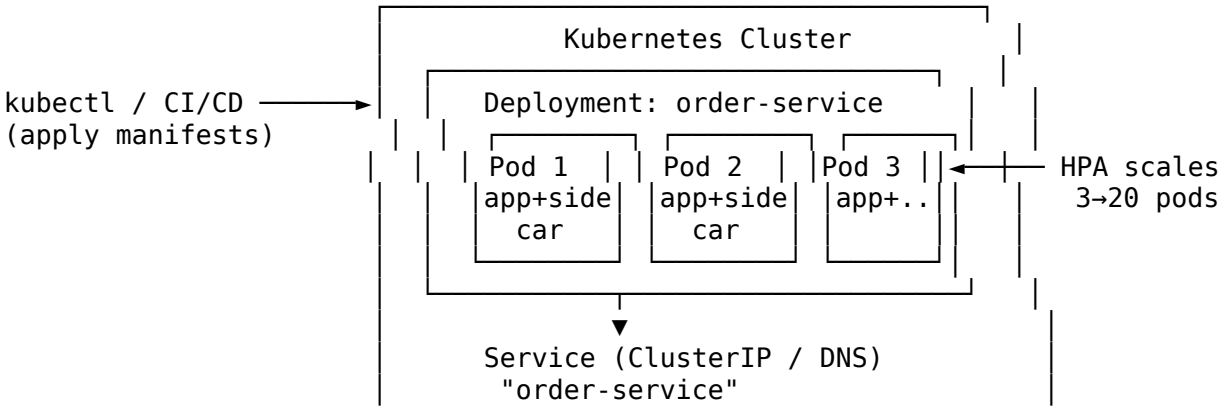
```java
@FeignClient(name = "order-service", url = "http://order-service")  // K8s
 ↪  Service DNS name
public interface OrderClient { ... }
```

3.5.3 Real-Time Usage

- Virtually every modern cloud-native company (**Spotify**, **Airbnb**, **CRED**) runs Kubernetes with one Deployment per microservice, HPA for auto-scaling during peak hours (e.g., IPL match streaming, festive sales).
- The **sidecar pattern** underlies service meshes like **Istio/Linkerd**, adding mTLS, retries, and traffic shaping without touching application code.
- Multi-stage Docker builds keep production images small (faster deploys, smaller attack surface) — standard practice at any company shipping containers to production.

Each microservice is independently containerized (Docker) and orchestrated (Kubernetes), sitting on the same event backbone:

3.5.4 Architectural Flow Diagram



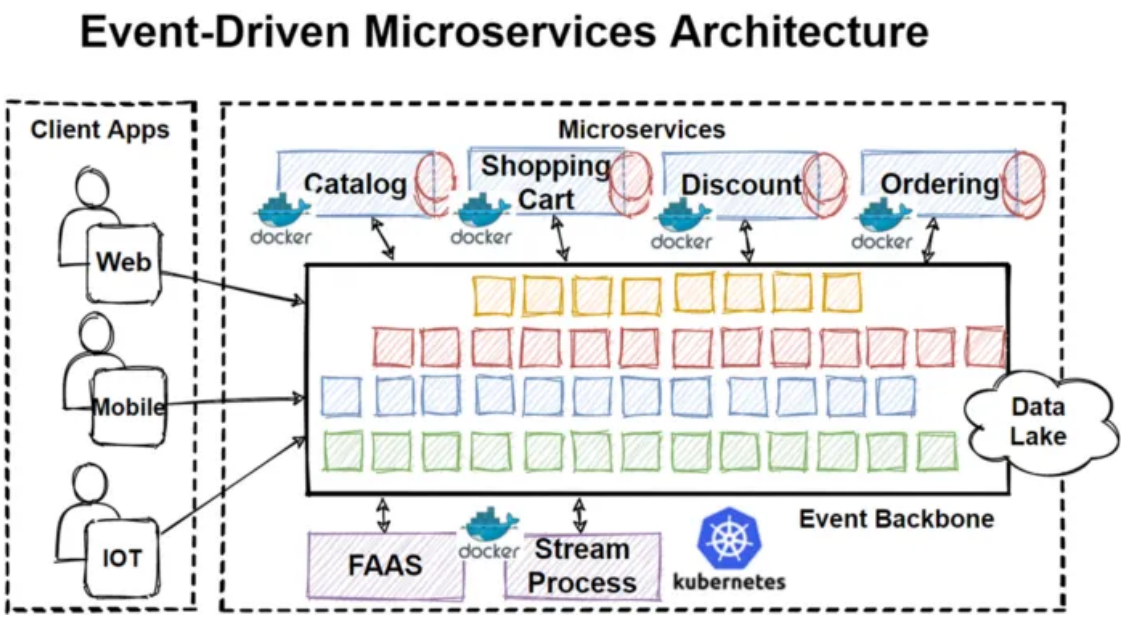


Figure 13: Event-Driven Microservices with Docker & Kubernetes

```
Other pods resolve via
Kubernetes DNS (built-in
service discovery)
```

4 Part III: Architecture Deep-Dives & Cross-Cutting Patterns

4.1 3-Tier Architecture

4.1.1 Definition

3-Tier Architecture separates a system into three physically distinct layers: **Presentation Tier** (UI/API layer), **Application/Business Logic Tier** (processes rules, orchestrates), and **Data Tier** (database). Unlike a monolith (one deployable), each tier can be deployed and scaled on its own hardware/VMs — this predates microservices and is still extremely common for traditional enterprise apps.

4.1.2 Hands-On: Runnable Mini-Project

Presentation Tier — React/Thymeleaf or a separate API layer (Spring Boot, stateless REST)

```
@RestController
@RequestMapping("/api/orders")
public class OrderPresentationController {
    private final OrderBusinessService businessService;   // calls tier 2 only

    @PostMapping
    public ResponseEntity<OrderResponse> create(@RequestBody OrderRequest req)
     ↪  {
        return ResponseEntity.ok(businessService.processOrder(req));
    }
}
```

Application/Business Tier — separate deployable, holds all business rules

```
@Service
public class OrderBusinessService {
    private final OrderDataClient dataClient;   // calls tier 3 over the
     ↪  network

    public OrderResponse processOrder(OrderRequest req) {
        validate(req);
        BigDecimal total = pricingEngine.calculate(req);
        OrderRecord saved = dataClient.save(new OrderRecord(req, total));
        return new OrderResponse(saved.getId(), total, "CONFIRMED");
    }
}
```

Data Tier — dedicated data-access service/DB, on its own servers

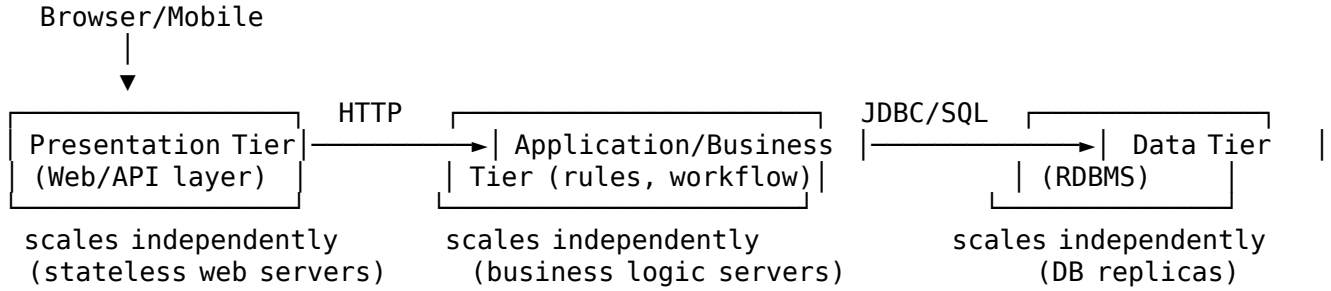
```
# deployed on separate infrastructure from tiers 1 & 2
services:
  data-tier-db:
    image: postgres:16
    deploy: { placement: { constraints: ["node.labels.tier==data"] } }
  app-tier:
    image: acme/order-business-service:1.0
```

```
    deploy: { placement: { constraints: ["node.labels.tier==app"] } }
  web-tier:
    image: acme/order-presentation-service:1.0
    deploy: { placement: { constraints: ["node.labels.tier==web"] } }
```

4.1.3 Real-Time Usage

- Traditional enterprise banking/insurance portals: web tier in a DMZ, app tier in a private subnet, DB tier locked down furthest from the internet — driven as much by **security zoning** as by scaling needs.
- Classic Java EE / .NET enterprise applications built before the microservices era, still running in many large enterprises today.

4.1.4 Architectural Flow Diagram



4.2 Microservices Architectures (Reference Architecture Deep-Dive)

4.2.1 Definition

Beyond individual services, a full **Microservices Architecture** needs a supporting platform: a **Config Server** (centralized externalized configuration), a **Service Registry** (who's alive, where), an **API Gateway** (edge), **Circuit Breakers** (resilience), and **Observability** (logs/metrics/traces). This is the "reference architecture" every mature microservices platform converges on, regardless of specific vendor choices.

4.2.2 Hands-On: Runnable Mini-Project

Spring Cloud Config Server — centralizes `application.yml` for every service in a Git repo

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { }
```

```
server: { port: 8888 }
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/acme/config-repo
          default-label: main
```

Every microservice pulls its config at startup

```
# order-service/bootstrap.yml
spring:
  application: { name: order-service }
  config:
    import: "configserver:http://config-server:8888"
```

Eureka Service Registry — services register themselves; the gateway discovers them dynamically

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication { }
```

```
# order-service registers with Eureka
eureka:
  client:
    service-url: { defaultZone: http://discovery-server:8761/eureka }
spring:
  application: { name: order-service }
```

Putting it together — a request's full journey through the platform

```
@RestController
public class OrderController {
    private static final Logger log =
     ↪  LoggerFactory.getLogger(OrderController.class);
```

```
    @PostMapping("/orders")
    @Timed(value = "orders.create.latency")     // Micrometer metric
    public OrderResponse create(@RequestBody OrderRequest req) {
        log.info("event=order_create_started productId={} qty={}",
↪   req.productId(), req.qty());
        OrderResponse resp = orderService.placeOrder(req);
        log.info("event=order_create_completed orderId={}", resp.orderId());
        return resp;
    }
}
```

4.2.3 Real-Time Usage

- Spring Cloud's **Config Server + Eureka + Gateway** trio was the de-facto Java reference stack from 2018-2022 and is still widely deployed; many shops have since migrated Eureka's role to **Kubernetes-native discovery** while keeping Config Server or moving to **HashiCorp Consul/Vault**.
- Netflix OSS (Eureka, Ribbon, Zuul, Hystrix) is the origin of this entire pattern family — literally built by Netflix to run its own streaming platform.

4.2.4 Architectural Flow Diagram

The generic reference shape — a gateway in front of a service mesh, a shared Management/Orchestration layer underneath, and DevOps continuously shipping changes to individual services:

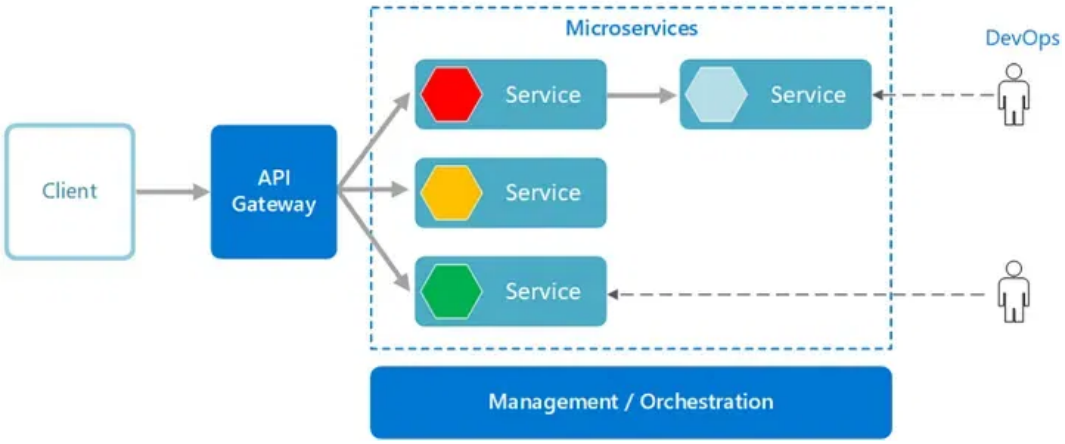
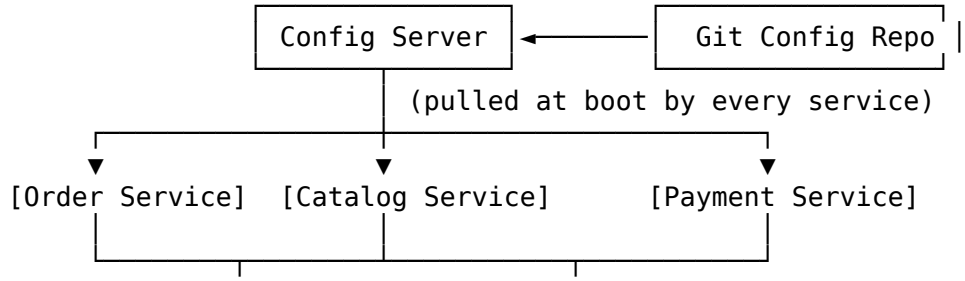
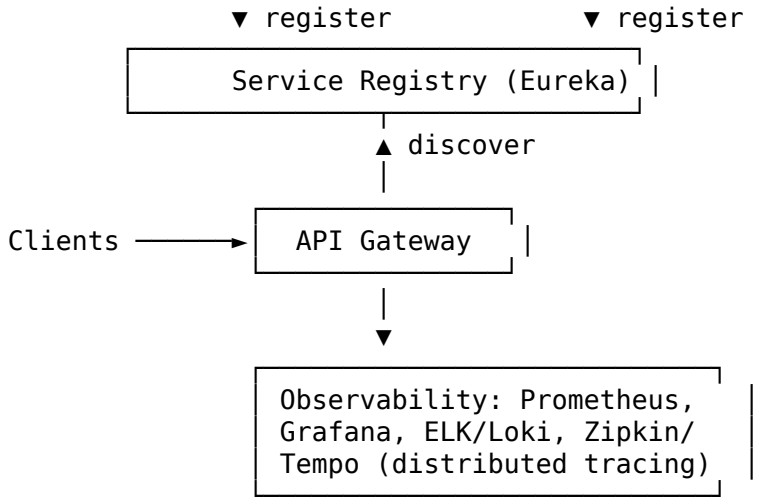


Figure 14: API Gateway, Microservices, Management/Orchestration & DevOps




▼ register
▼ register
Service Registry (Eureka)
▲ discover
Clients
API Gateway
Observability: Prometheus,
Grafana, ELK/Loki, Zipkin/
Tempo (distributed tracing)

4.3 Microservices Architectures — API Gateways (Deep-Dive)

4.3.1 Definition

An **API Gateway** is the single entry point for all external clients into a microservices system. Beyond simple routing, a production gateway handles: **request aggregation** (combine multiple backend calls into one client response), **protocol translation** (REST-in, gRPC-out), **authentication/authorization**, **rate limiting/quotas**, and **response caching**. Popular implementations: Spring Cloud Gateway, Kong, AWS API Gateway, NGINX.

4.3.2 Hands-On: Runnable Mini-Project

Request Aggregation — one client call fans out to 3 services and merges the response

```
@RestController
public class OrderDetailAggregatorController {

    private final OrderClient orderClient;
    private final CustomerClient customerClient;
    private final ShippingClient shippingClient;

    @GetMapping("/api/order-details/{orderId}")
    public Mono<OrderDetailResponse> getOrderDetail(@PathVariable Long
     ↪  orderId) {
        Mono<OrderView> order = orderClient.getOrder(orderId);
        Mono<CustomerView> customer = order.flatMap(o ->
 ↪  customerClient.getCustomer(o.customerId()));
        Mono<ShipmentView> shipment = shippingClient.getShipment(orderId);

        return Mono.zip(order, customer, shipment)
                .map(tuple -> new OrderDetailResponse(tuple.getT1(),
                 ↪  tuple.getT2(), tuple.getT3()));
    }
}
```

Gateway route configuration with rewriting, timeout, and retry

```
spring:
  cloud:
    gateway:
      routes:
        - id: catalog-service
          uri: lb://catalog-service
          predicates: [ "Path=/api/v1/catalog/**" ]
          filters:
            - RewritePath=/api/v1/catalog/(?<segment>.*), /$\{segment}
            - name: Retry
              args: { retries: 3, statuses: BAD_GATEWAY, methods: GET }
            - name: RequestSize
              args: { maxSize: 5MB }
      httpclient:
        connect-timeout: 2000
        response-timeout: 5s
```

Rate limiting per API key using Redis-backed token bucket

```
@Bean
KeyResolver apiKeyResolver() {
    return exchange -> Mono.justOrEmpty(
        exchange.getRequest().getHeaders().getFirst("X-API-KEY"));
}
```

4.3.3 Real-Time Usage

- **Netflix's** gateway aggregates dozens of backend calls into a single payload optimized per device (phone, TV, browser) — reducing client round-trips over slow mobile networks.
- **Kong**/AWS API Gateway commonly front public SaaS APIs, enforcing per-customer rate limits tied to subscription tier (Free/Pro/Enterprise quotas).
- Financial APIs use the gateway layer for mandatory request signing/verification before anything reaches internal services.

4.3.4 Architectural Flow Diagram

The gateway as the single entry point in front of every backend service:

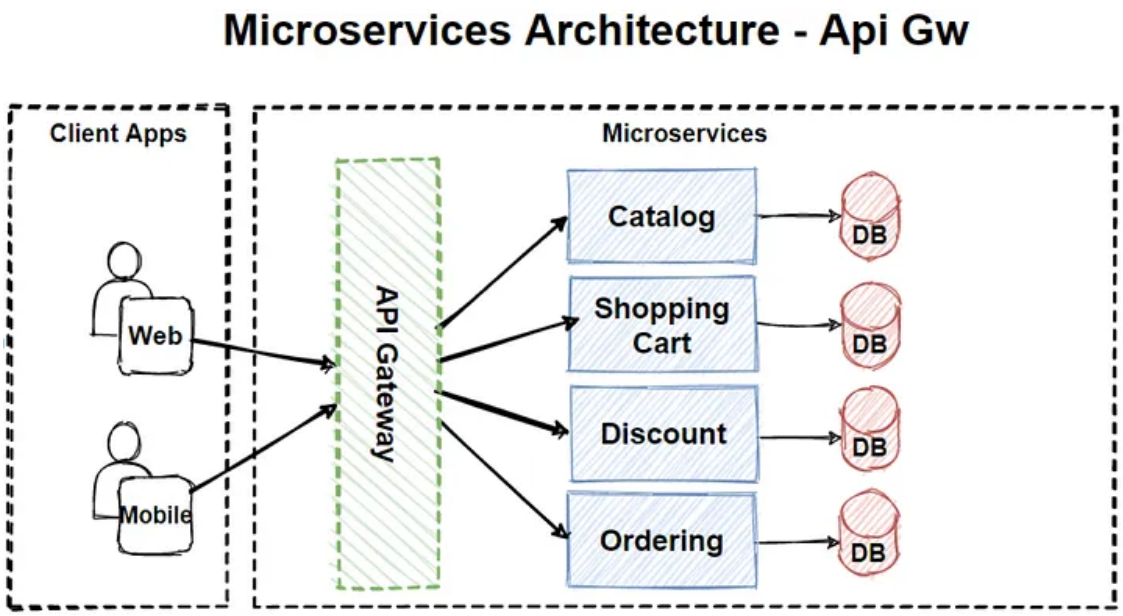
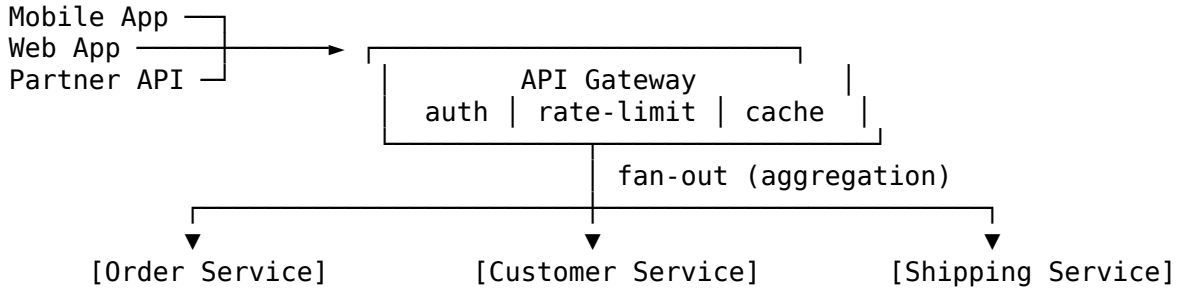
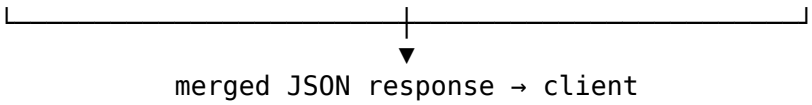


Figure 15: Microservices Architecture — API Gateway

Request aggregation view (fan-out/fan-in through the gateway):




merged JSON response → client

4.4 Microservices Architectures — API Gateway (BFF: Backend For Frontend)

4.4.1 Definition

BFF (Backend For Frontend) takes the API Gateway idea further: instead of *one* generic gateway serving every client type, you build **one gateway per client experience** (Mobile-BFF, Web-BFF, Partner-BFF). Each BFF shapes/aggregates responses exactly the way that specific frontend needs, so mobile apps aren't forced to parse the same bloated payload a desktop web app receives.

4.4.2 Hands-On: Runnable Mini-Project

Mobile-BFF — minimal payload, optimized for bandwidth

```
@RestController
@RequestMapping("/mobile-bff")
public class MobileBffController {
    @GetMapping("/orders/{id}")
    public MobileOrderView getOrder(@PathVariable Long id) {
        OrderView order = orderClient.getOrder(id);
        // Only the fields a mobile screen actually renders
        return new MobileOrderView(order.id(), order.status(),
         ↪  order.totalAmount());
    }
}
```

Web-BFF — richer payload, includes data the desktop UI needs (order history, recommendations)

```
@RestController
@RequestMapping("/web-bff")
public class WebBffController {
    @GetMapping("/orders/{id}")
    public WebOrderView getOrder(@PathVariable Long id) {
        OrderView order = orderClient.getOrder(id);
        List<ProductView> recommendations =
         ↪  recommendationClient.getFor(order.customerId());
        List<OrderView> history =
         ↪  orderClient.getHistoryFor(order.customerId());
        return new WebOrderView(order, recommendations, history);
    }
}
```

Each BFF is its own deployable, independently scaled Spring Boot service

```
services:
  mobile-bff: { build: ./mobile-bff, ports: ["8091:8080"] }
  web-bff:    { build: ./web-bff,    ports: ["8092:8080"] }
  partner-bff:{ build: ./partner-bff,ports: ["8093:8080"] }
```

4.4.3 Real-Time Usage

- **SoundCloud** and **Netflix** popularized BFF: each device team (iOS, Android, TV, Web) owns its own BFF, decoupling frontend release cycles from backend service changes.

- E-commerce apps use a lightweight Mobile-BFF to hit performance budgets on 3G/4G networks while the Web-BFF can afford heavier payloads over broadband.
- **Microsoft's eShopOnContainers** reference application is a widely-studied real-world BFF implementation: an **Ocelot** API gateway plus a dedicated **Aggregator** service front Catalog, Basket, Discount, and Ordering microservices — each with its own polyglot store (MongoDB, Redis, PostgreSQL, SQL Server) behind RabbitMQ:

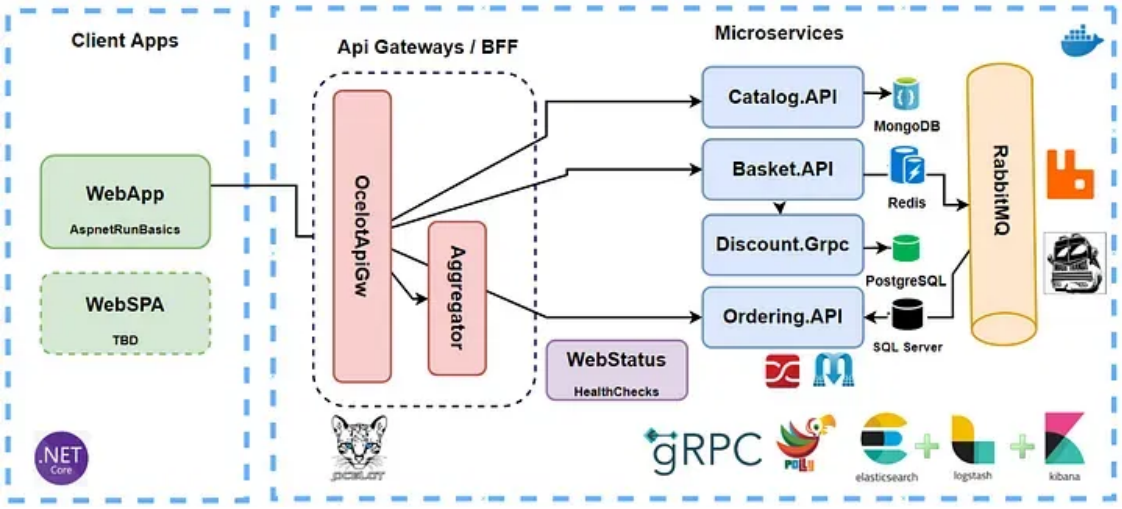


Figure 16: Api Gateways / BFF — Ocelot + Aggregator (eShopOnContainers reference)

The same reference app taken to its full scale shows **separate BFFs per client AND per concern** (`Mobile-Shopping`, `Mobile-Marketing`, `Web-Shopping`, `Web-Marketing`), fronting six independently-stored microservices behind a shared Event Bus, all running on a Docker host:

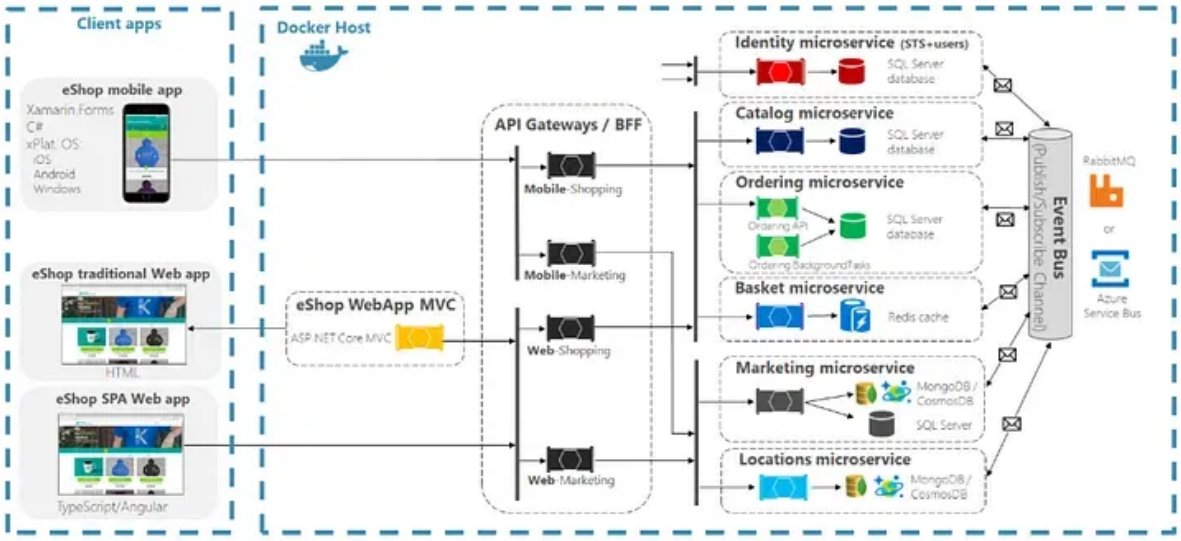


Figure 17: eShopOnContainers — Full BFF + Microservices + Event Bus Reference Architecture

4.4.4 Architectural Flow Diagram

Generic BFF pattern — each client type gets its own dedicated gateway:

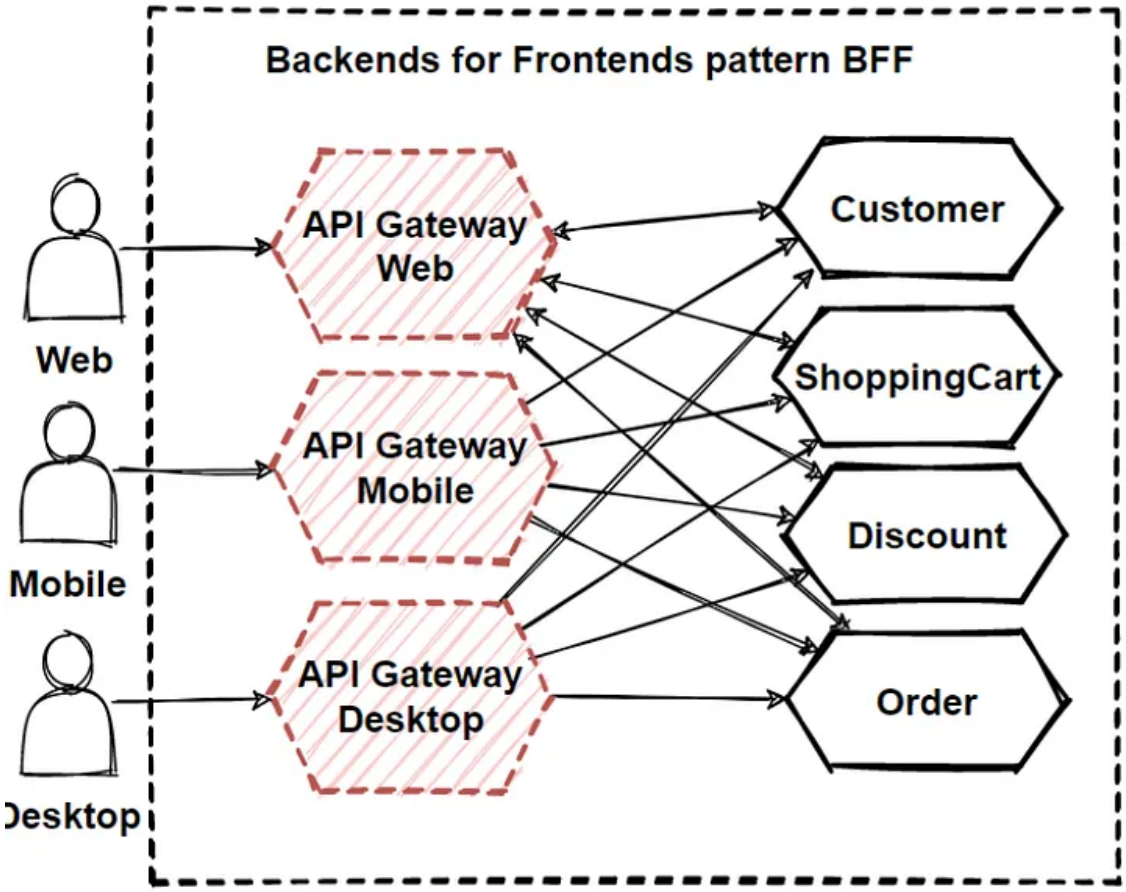


Figure 18: Backends for Frontends Pattern (BFF)

Applied to our e-commerce platform — Web/Mobile/Desktop BFFs fronting the same downstream services:

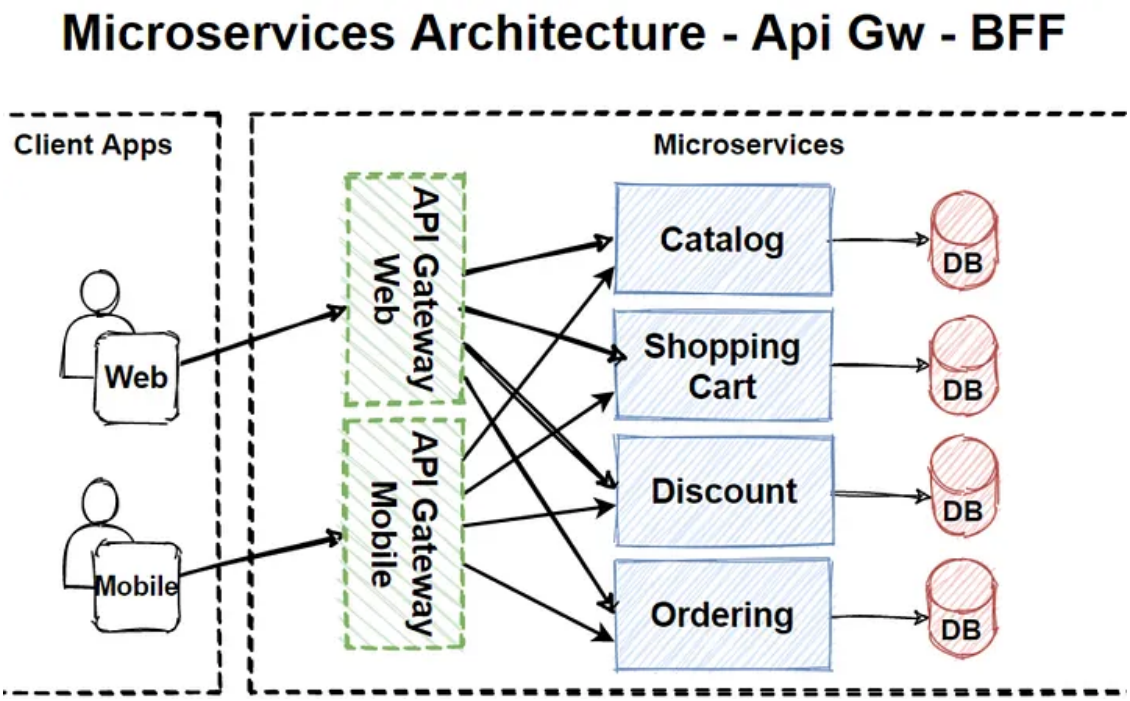


Figure 19: Microservices Architecture — API Gateway — BFF

4.5 Microservices Architecture — Service Aggregator / Registry Patterns

4.5.1 Definition

The **Aggregator Pattern** collects data from multiple microservices and combines it into a single response (can live in the gateway or as its own dedicated service). The **Service Registry pattern** (Client-Side or Server-Side Discovery) lets services find each other dynamically at runtime instead of hardcoding IPs — every instance **registers** itself on startup and sends **heartbeats**; consumers query the registry (or let a load balancer/router query it) to find healthy instances.

4.5.2 Hands-On: Runnable Mini-Project

Dedicated Aggregator Service (separate from the gateway) — useful when aggregation logic is complex/reusable

```
@Service
public class OrderAggregatorService {
    private final OrderClient orderClient;
    private final PaymentClient paymentClient;
    private final ShippingClient shippingClient;

    public CompletableFuture<FullOrderView> aggregate(Long orderId) {
        CompletableFuture<OrderView> order =
 ↪  orderClient.getOrderAsync(orderId);
        CompletableFuture<PaymentView> payment =
 ↪  paymentClient.getPaymentAsync(orderId);
        CompletableFuture<ShipmentView> shipment =
 ↪  shippingClient.getShipmentAsync(orderId);

        return CompletableFuture.allOf(order, payment, shipment)
                .thenApply(v -> new FullOrderView(order.join(), payment.join(),
                  ↪  shipment.join()));
    }
}
```

Server-Side Discovery — Eureka registry, client only knows the logical service name

```
@EnableDiscoveryClient
@SpringBootApplication
public class OrderServiceApplication { }
```

```
eureka:
  instance:
    lease-renewal-interval-in-seconds: 10     # heartbeat frequency
    lease-expiration-duration-in-seconds: 30  # evict if no heartbeat
  client:
    service-url: { defaultZone: http://discovery-server:8761/eureka }
```

Client-Side Discovery — client library (Spring Cloud LoadBalancer) picks an instance itself

```
@LoadBalanced
@Bean
```

```java
public RestTemplate restTemplate() { return new RestTemplate(); }

// "inventory-service" resolved via the registry + client-side round-robin
 ↪  load balancing
restTemplate.getForObject("http://inventory-service/stock/{id}",
 ↪  StockResponse.class, id);
```

4.5.3 Real-Time Usage

- Kubernetes itself implements **server-side discovery** natively (kube-proxy + Services), which is why most cloud-native shops no longer run Eureka separately.
- Aggregator services are common in **travel-booking platforms** (flight + hotel + car-rental results merged into one search response) and **dashboard/BI backends**.

4.5.4 Architectural Flow Diagram

The aggregator pattern in isolation:

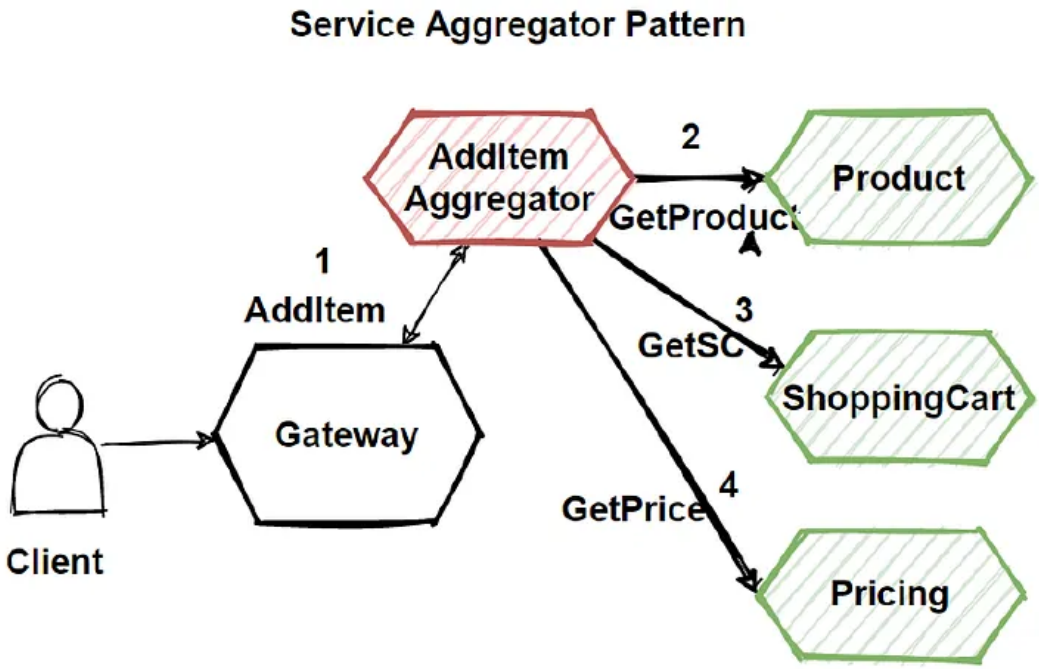


Figure 20: Service Aggregator Pattern

Applied at the platform level, alongside a service registry:

A fuller picture — Service Discovery paired with a separate Management plane, plus an Identity Provider and CDN/static-content path that never touches the service layer at all:

```
┌──────────────────┐
│ Service Registry │◄──heartbeat── [Order Svc instance 1,2,3]
│  (Eureka/Consul) │◄──heartbeat── [Payment Svc instance 1,2]
└──────────────────┘
          ▲ query "who is healthy?"
          │
```

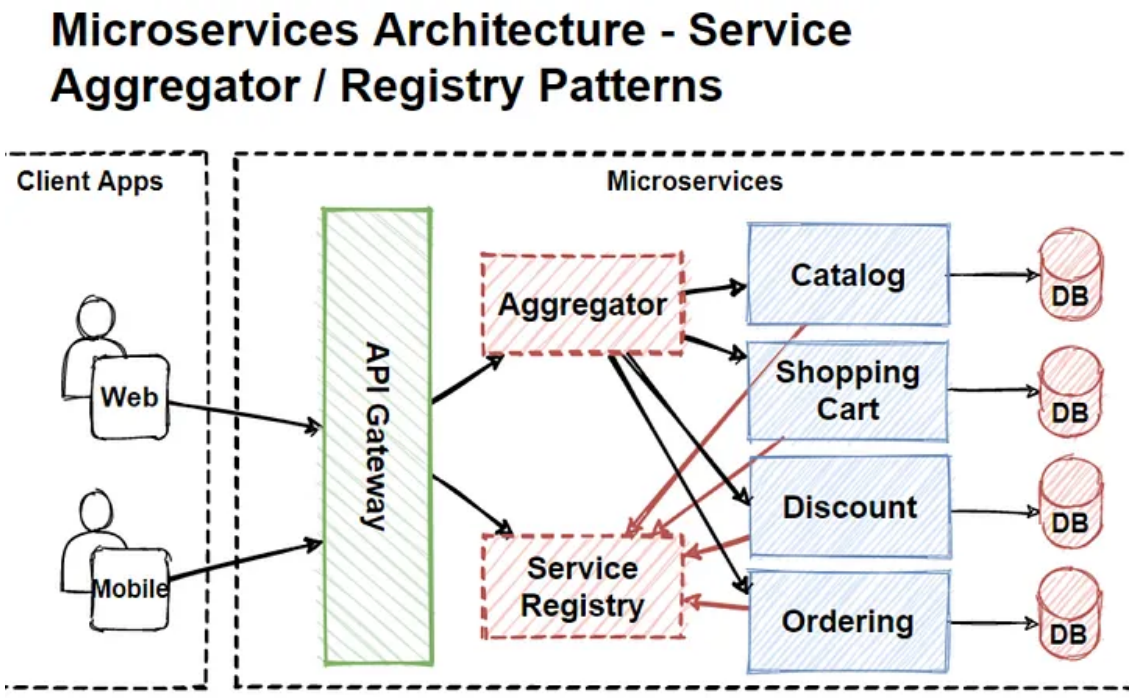


Figure 21: Microservices Architecture — Service Aggregator / Registry Patterns

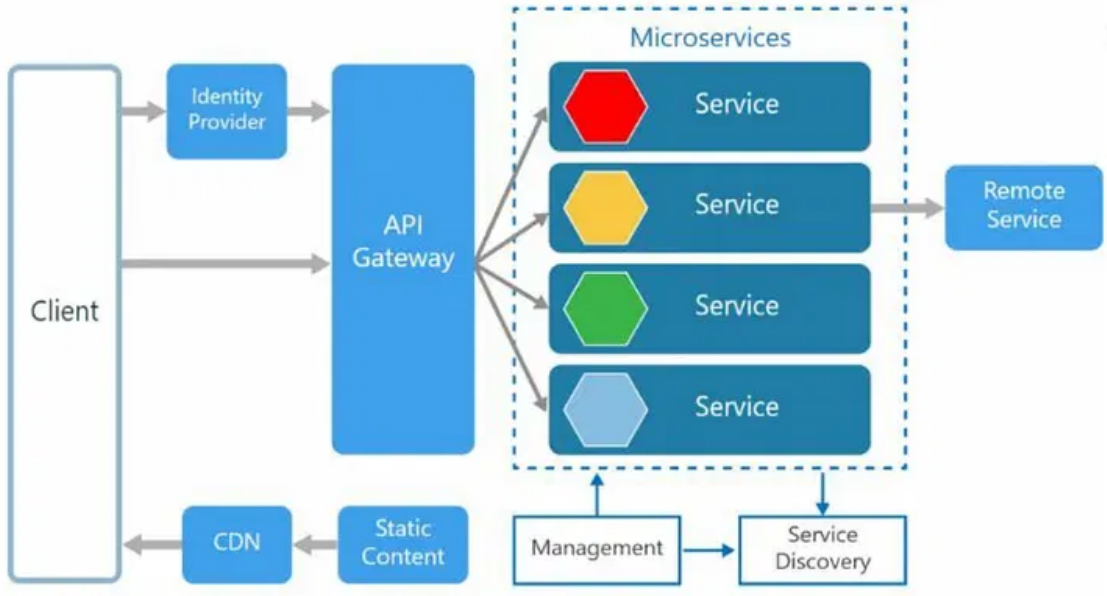
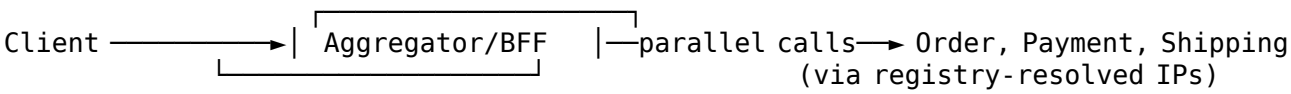


Figure 22: API Gateway with Identity Provider, Service Discovery & Management


Client
Aggregator/BFF
parallel calls
Order, Payment, Shipping
(via registry-resolved IPs)

4.6 Microservices Architecture — Message Broker

4.6.1 Definition

A **Message Broker** (Kafka, RabbitMQ) is the intermediary that decouples producers from consumers: producers publish messages to a **topic/queue**, and the broker durably stores and delivers them to one or more consumers, handling buffering, ordering (per-partition in Kafka), retries, and dead-lettering. Two delivery models: **Point-to-point (queue)** — one consumer processes each message (RabbitMQ classic queues), and **Publish/Subscribe (topic)** — every subscriber gets its own copy (Kafka consumer groups, one copy per group).

A broker often sits at the center of an entire integration landscape, not just microservices — any sink or source can plug in:

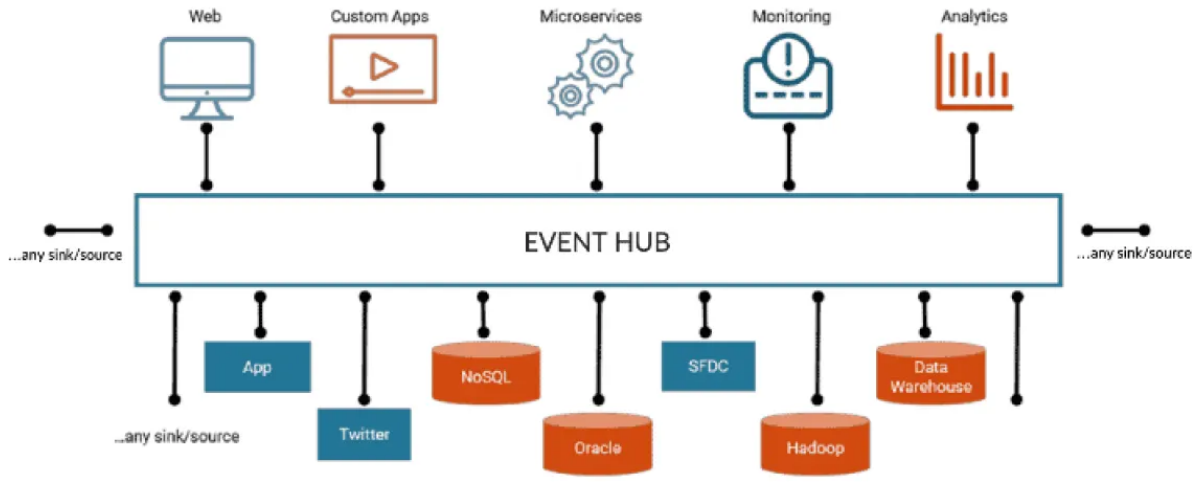


Figure 23: Event Hub

4.6.2 Hands-On: Runnable Mini-Project

Kafka producer with delivery guarantees + idempotency

```
@Configuration
public class KafkaProducerConfig {
    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> props = new HashMap<>();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092");
        props.put(ProducerConfig.ACKS_CONFIG, "all");                    // wait
 ↪  for all ISR replicas
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);      //
 ↪  exactly-once producer
        props.put(ProducerConfig.RETRIES_CONFIG, 5);
        return new DefaultKafkaProducerFactory<>(props);
    }
}
```

Consumer with manual ack + dead-letter topic for poison messages

```
@KafkaListener(topics = "order.placed", groupId = "inventory-service",
               containerFactory = "manualAckContainerFactory")
```

```
public void consume(ConsumerRecord<String, OrderPlacedEvent> record,
 ↪  Acknowledgment ack) {
    try {
        inventoryService.reserve(record.value());
        ack.acknowledge();                      // commit offset only after
 ↪  success
    } catch (Exception ex) {
        deadLetterPublisher.send("order.placed.DLT", record.value());  //
 ↪  poison message
        ack.acknowledge();                      // don't block the partition
 ↪  forever
    }
}
```

RabbitMQ alternative — point-to-point queue with a competing-consumers pattern

```
@RabbitListener(queues = "order.processing.queue", concurrency = "5-10")
public void process(OrderMessage message, Channel channel,
                    @Header(AmqpHeaders.DELIVERY_TAG) long tag) throws
                     ↪  IOException {
    try {
        processOrder(message);
        channel.basicAck(tag, false);
    } catch (Exception e) {
        channel.basicNack(tag, false, true);   // requeue for retry
    }
}
```

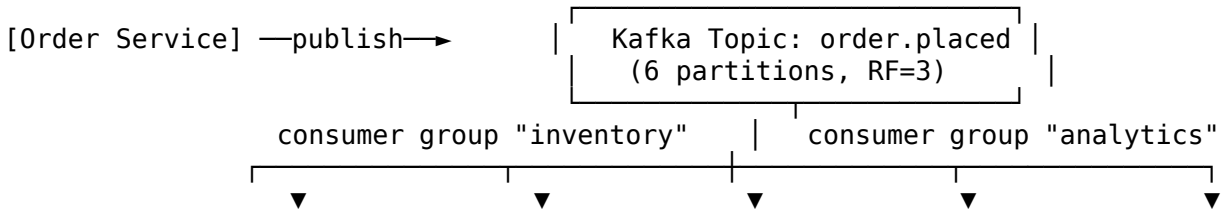
Docker Compose — Kafka cluster with 3 partitions/topic for parallelism

```
# create-topics.sh
kafka-topics --create --topic order.placed --partitions 6 --replication-factor
 ↪  3 \
  --bootstrap-server kafka:9092
```

4.6.3 Real-Time Usage

- **LinkedIn** (Kafka's birthplace) processes trillions of messages/day for activity streams, metrics pipelines, and service integration.
- RabbitMQ's point-to-point queues are common for **task queues** — image processing, PDF generation, background jobs — where exactly one worker should handle each job.
- Dead-letter topics/queues are standard practice everywhere so a single malformed message ("poison pill") can't stall an entire partition/queue.

4.6.4 Architectural Flow Diagram



```
                  [Inventory-1]    [Inventory-2]    [Inventory-3]   [Analytics-
1]  [Analytics-2]
           (each partition consumed by exactly one instance within its own group)
                     │ on failure
                     ▼
              order.placed.DLT  (dead-letter topic for poison messages)
```

4.7 Microservices Architecture — Database Sharding (Cassandra)

4.7.1 Definition

Sharding splits a dataset horizontally across multiple nodes so no single node holds all the data, enabling near-linear write/read scalability. **Apache Cassandra** is built for this natively: every table has a **partition key** that determines which node(s) own a row via consistent hashing, plus **clustering columns** that order rows within a partition. Replication factor (RF) controls how many nodes hold a copy, and **tunable consistency** (ONE, QUORUM, ALL) lets you trade latency for consistency per query.

4.7.2 Hands-On: Runnable Mini-Project

Cassandra keyspace + table design for an "Order Events" write-heavy log (billions of rows)

```
CREATE KEYSPACE ecommerce WITH replication = {
  'class': 'NetworkTopologyStrategy', 'datacenter1': 3
};

CREATE TABLE ecommerce.order_events_by_customer (
  customer_id    UUID,
  event_time     TIMESTAMP,
  order_id       UUID,
  event_type     TEXT,
  payload        TEXT,
  PRIMARY KEY ((customer_id), event_time)
) WITH CLUSTERING ORDER BY (event_time DESC);
-- partition key = customer_id -> determines the shard/node
-- clustering key = event_time -> rows within a partition sorted newest-first
```

Spring Boot + Spring Data Cassandra repository

```
@Table("order_events_by_customer")
public class OrderEvent {
    @PrimaryKeyColumn(name = "customer_id", type = PrimaryKeyType.PARTITIONED)
    private UUID customerId;
    @PrimaryKeyColumn(name = "event_time", type = PrimaryKeyType.CLUSTERED,
     ↪  ordering = Ordering.DESCENDING)
    private Instant eventTime;
    private UUID orderId;
    private String eventType;
    private String payload;
}

public interface OrderEventRepository extends CassandraRepository<OrderEvent,
 ↪  UUID> {
    List<OrderEvent> findByCustomerId(UUID customerId);   // single-partition
     ↪  read = fast
}
```

Writing with tunable consistency (QUORUM for important writes)

```
@Service
public class OrderEventService {
    private final CassandraTemplate cassandraTemplate;
```

```
    public void record(OrderEvent event) {
        Insert insert = QueryBuilder.insertInto("order_events_by_customer")
                .value("customer_id", event.getCustomerId())
                .value("event_time", event.getEventTime())
                .value("order_id", event.getOrderId())
                .value("event_type", event.getEventType())
                .value("payload", event.getPayload());
        insert.setConsistencyLevel(ConsistencyLevel.QUORUM);   // RF=3 ->
 ↪  needs 2 acks
        cassandraTemplate.getCqlOperations().execute(insert);
    }
}
```

application.yml

```
spring:
  data:
    cassandra:
      keyspace-name: ecommerce
      contact-points: cassandra-node1,cassandra-node2,cassandra-node3
      local-datacenter: datacenter1
      consistency-level: quorum
```

Anti-pattern warning — designing by query, not by "normalized entities"

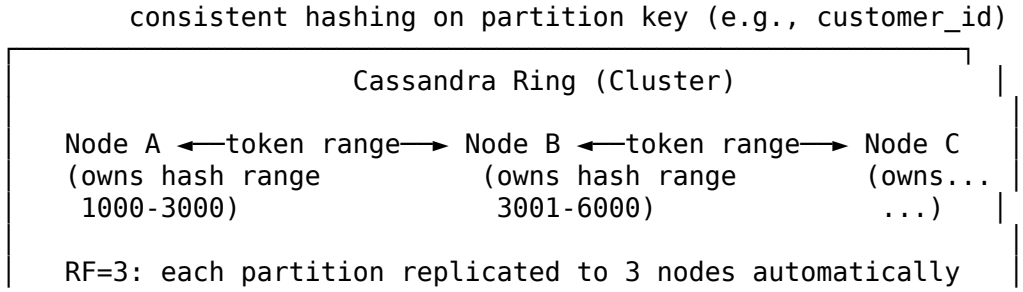
```
WRONG (relational thinking): one giant "orders" table, then filter by customer_id with a WHERE
   clause and secondary index → fans out across ALL nodes, slow at scale.

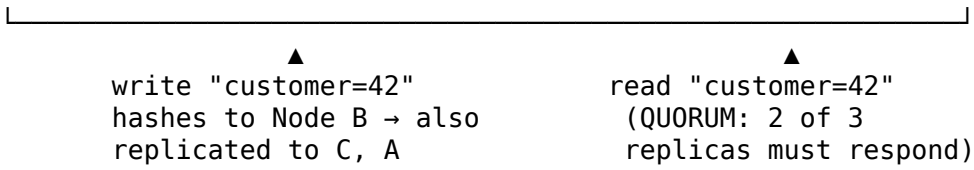
RIGHT (Cassandra thinking): model a dedicated table PER QUERY PATTERN
   - order_events_by_customer   (for "show me this customer's history")
   - order_events_by_order_id   (for "show me this order's timeline")
    Denormalize/duplicate data across tables; disk is cheap, cross-node fan-
out is expensive.
```

4.7.3 Real-Time Usage

- **Netflix** uses Cassandra for viewing-history and personalization data — petabyte scale, multi-region active-active writes.
- **Instagram/Discord**-style messaging systems shard by `conversation_id`/`channel_id` so all messages for a chat land on the same partition for fast reads.
- IoT/telemetry pipelines shard by `device_id` for consistent per-device write throughput.

4.7.4 Architectural Flow Diagram




write "customer=42"
hashes to Node B → also
replicated to C, A
read "customer=42"
(QUORUM: 2 of 3
replicas must respond)

4.8 Event-Driven Microservices Architecture (Deep-Dive: Event Sourcing & Streaming)

4.8.1 Definition

Going beyond simple pub/sub (Part I), a mature Event-Driven Architecture often adopts **Event Sourcing**: instead of storing only the *current state* of an entity, you store the **full sequence of events** that led to it — the event log *is* the source of truth, and current state is derived by replaying events. Combined with **Kafka Streams** for real-time stream processing (windowed aggregations, joins between streams), this enables both auditability and live analytics.

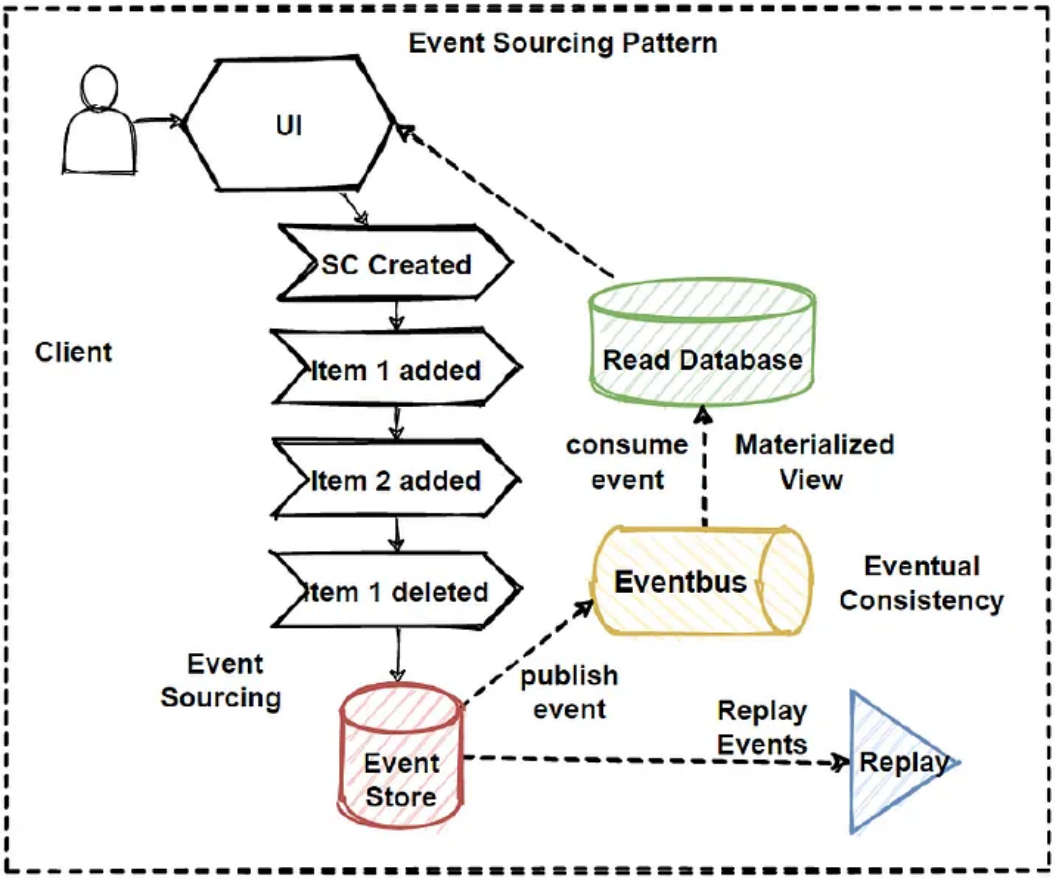


Figure 24: Event Sourcing Pattern

4.8.2 Hands-On: Runnable Mini-Project

Event-sourced `Order` aggregate — state is rebuilt by folding over events, never stored directly

```
public class Order {
    private Long id;
    private OrderStatus status;
    private BigDecimal total;
    private final List<Object> uncommittedEvents = new ArrayList<>();

    public static Order place(Long id, BigDecimal total) {
```

```
        Order order = new Order();
        order.apply(new OrderPlacedEvent(id, total, Instant.now()));
        return order;
    }

    public void cancel(String reason) {
        if (status != OrderStatus.PLACED) throw new
         ↪  IllegalStateException("Cannot cancel");
        apply(new OrderCancelledEvent(id, reason, Instant.now()));
    }

    // Rebuild state by replaying persisted events (used when loading from the
     ↪  event store)
    public static Order replay(List<Object> history) {
        Order order = new Order();
        history.forEach(order::apply);
        order.uncommittedEvents.clear();
        return order;
    }

    private void apply(Object event) {
        if (event instanceof OrderPlacedEvent e) { this.id = e.orderId();
         ↪  this.total = e.total(); this.status = OrderStatus.PLACED; }
        if (event instanceof OrderCancelledEvent e) { this.status =
         ↪  OrderStatus.CANCELLED; }
        uncommittedEvents.add(event);
    }
}
```

Event Store persistence (append-only, Kafka as the durable event log)

```
@Service
public class EventStore {
    private final KafkaTemplate<String, Object> kafka;

    public void save(Order order, List<Object> newEvents) {
        newEvents.forEach(event ->
            kafka.send("order-events", order.getId().toString(), event));  //
 ↪  append-only, never update/delete
    }

    public Order load(Long orderId) {
        List<Object> history = readAllEventsForKey("order-events",
         ↪  orderId.toString()); // from a compacted/replayed topic
        return Order.replay(history);
    }
}
```

Kafka Streams — real-time windowed aggregation (e.g., orders-per-minute per product, for a live dashboard)

```
@Bean
public KStream<String, OrderPlacedEvent> ordersPerMinuteStream(StreamsBuilder
 ↪  builder) {
```

```
    KStream<String, OrderPlacedEvent> stream = builder.stream("order.placed");

    stream.groupBy((key, event) -> event.productId().toString())
          .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
          .count()
          .toStream()
          .foreach((windowedKey, count) ->
              log.info("product={} window={} orderCount={}",
                       windowedKey.key(), windowedKey.window(), count));
    return stream;
}
```

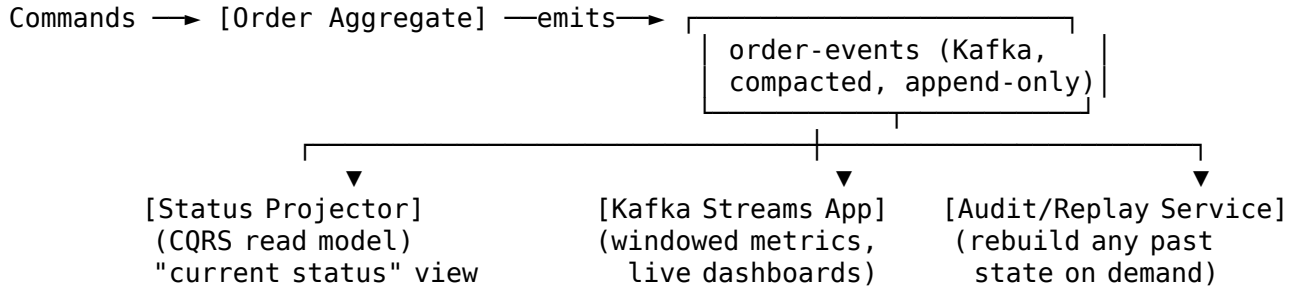
CQRS read-side rebuilt from the same event stream (materialized "current order status" view)

```
@KafkaListener(topics = "order-events", groupId = "order-status-projector")
public void project(Object event) {
    if (event instanceof OrderPlacedEvent e)
     ↪  statusViewRepo.upsert(e.orderId(), "PLACED");
    if (event instanceof OrderCancelledEvent e)
     ↪  statusViewRepo.upsert(e.orderId(), "CANCELLED");
}
```

4.8.3 Real-Time Usage

- Banking ledgers are the canonical Event Sourcing example: never overwrite a balance — append `Debited`/`Credited` events, derive current balance by replay, giving a perfect audit trail for compliance.
- **Uber's** and **DoorDash's** real-time ETA/surge-pricing dashboards run on Kafka Streams windowed aggregations over ride/order event streams.
- Any "undo/redo" or "time travel debugging" feature (e.g., in collaborative editors) is naturally modeled with event sourcing.

4.8.4 Architectural Flow Diagram



4.9 Microservices Distributed Caching (Caffeine, Redis, Cache Eviction: LRU, LFU, TTL)

4.9.1 Definition

Production caching typically layers **two tiers**: an **L1 local in-JVM cache** (Caffeine — nanosecond access, but per-instance, not shared) and an **L2 distributed cache** (Redis — shared across all instances, survives a single pod restart). **Eviction policies** decide what to remove when the cache is full: **LRU** (Least Recently Used — evict the item not accessed in the longest time), **LFU** (Least Frequently Used — evict the item accessed the fewest times), and **TTL** (Time-To-Live — evict after a fixed expiry regardless of access pattern).

4.9.2 Hands-On: Runnable Mini-Project

L1: Caffeine local cache with size-based LRU-like eviction + TTL

```
@Bean
public Cache<Long, ProductView> productLocalCache() {
    return Caffeine.newBuilder()
            .maximumSize(10_000)                    // evict via Window-TinyLFU
             ↪  when full
            .expireAfterWrite(Duration.ofMinutes(5))  // TTL
            .recordStats()
            .build();
}

@Service
public class ProductService {
    private final Cache<Long, ProductView> localCache;
    private final RedisTemplate<String, ProductView> redis;
    private final ProductRepository repo;

    public ProductView getProduct(Long id) {
        // L1 check first (fastest, no network hop)
        ProductView cached = localCache.getIfPresent(id);
        if (cached != null) return cached;

        // L2 check (shared across all pods)
        ProductView fromRedis = redis.opsForValue().get("product:" + id);
        if (fromRedis != null) {
            localCache.put(id, fromRedis);
            return fromRedis;
        }

        // L3: DB, then populate both cache tiers
        ProductView fromDb = toView(repo.findById(id).orElseThrow());
        redis.opsForValue().set("product:" + id, fromDb,
 ↪  Duration.ofMinutes(30));
        localCache.put(id, fromDb);
        return fromDb;
    }
}
```

Redis configured for LFU eviction (good for “hot key” workloads like trending products)

```
# redis.conf
maxmemory 512mb
maxmemory-policy allkeys-lfu    # evict least-frequently-used keys when memory is full
```

Redis configured for LRU (good for generic session/page caches)

```
maxmemory-policy allkeys-lru
```

Explicit TTL-only eviction for time-sensitive data (e.g., OTP codes, price quotes)

```
redis.opsForValue().set("otp:" + userId, otpCode, Duration.ofMinutes(5));  //
 ↪  hard expiry, no LRU/LFU involved
```

Cache stampede protection (many requests miss simultaneously on a hot key expiry)

```
@Service
public class StampedeSafeCache {
    private final RedissonClient redisson;   // provides distributed locks

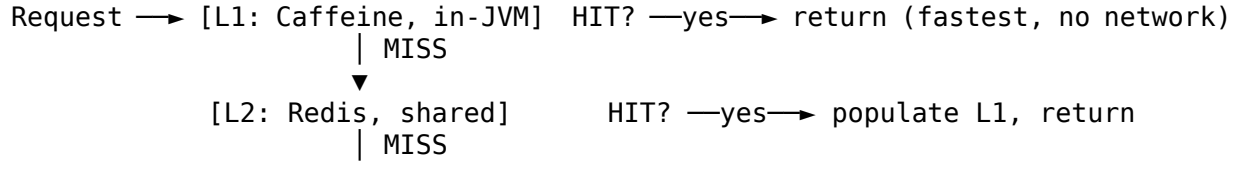
    public ProductView getWithLock(Long id) {
        String cacheKey = "product:" + id;
        ProductView cached = redis.opsForValue().get(cacheKey);
        if (cached != null) return cached;

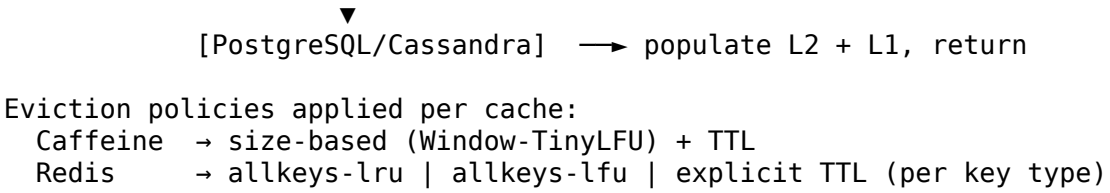
        RLock lock = redisson.getLock("lock:" + cacheKey);
        lock.lock();
        try {
            cached = redis.opsForValue().get(cacheKey);      // double-check
 ↪  after acquiring lock
            if (cached != null) return cached;
            ProductView fresh =
 ↪  repo.findById(id).map(this::toView).orElseThrow();
            redis.opsForValue().set(cacheKey, fresh, Duration.ofMinutes(10));
            return fresh;
        } finally {
            lock.unlock();
        }
    }
}
```

4.9.3 Real-Time Usage

- **Twitter/X** uses LRU-style local caches for timelines and LFU-style caches for trending-topics counters (frequency matters more than recency there).
- Two-tier (Caffeine + Redis) caching is standard in high-QPS Java services — Caffeine absorbs 90%+ of reads with zero network cost, Redis is the fallback shared layer.
- OTP/session-token stores universally use TTL-only eviction — correctness (must expire on schedule) matters more than memory pressure.

4.9.4 Architectural Flow Diagram




▼
[PostgreSQL/Cassandra] ⟶ populate L2 + L1, return
Eviction policies applied per cache:
Caffeine → size-based (Window-TinyLFU) + TTL
Redis → allkeys-lru | allkeys-lfu | explicit TTL (per key type)

4.10 Microservices Containers and Orchestrators with CI/CD (Canary, Blue-Green, Logs, Metrics)

4.10.1 Definition

A **CI/CD Pipeline** automates build → test → containerize → deploy. Two safe-release strategies on Kubernetes: **Blue-Green** — run two full environments (Blue=current, Green=new); switch all traffic atomically once Green is verified, instant rollback by switching back. **Canary** — release the new version to a small % of traffic first, watch metrics/error rates, then progressively ramp to 100% (or auto-rollback on regression). Both rely on strong **Logs** (structured, centralized) and **Metrics** (Prometheus/Grafana) to make the go/no-go decision.

4.10.2 Hands-On: Runnable Mini-Project

GitHub Actions CI/CD pipeline — build, test, containerize, push, deploy

```yaml
name: order-service-ci-cd
on: { push: { branches: [main] } }
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - run: mvn -B test
      - run: mvn -B package -DskipTests

  containerize-push:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: registry.acme.com/order-service:${{ github.sha }}

  deploy-canary:
    needs: containerize-push
    runs-on: ubuntu-latest
    steps:
      - run: |
          kubectl set image deployment/order-service-canary \
            order-service=registry.acme.com/order-service:${{ github.sha }}
          kubectl argo rollouts promote order-service --namespace prod
```

Blue-Green with Kubernetes Services (instant switch by changing the selector)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service-blue }
spec:
  replicas: 5
  selector: { matchLabels: { app: order-service, version: blue } }
  template:
```

```
    metadata: { labels: { app: order-service, version: blue } }
    spec: { containers: [{ name: order-service, image:
 ↪  registry.acme.com/order-service:1.4.0 }] }
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service-green }
spec:
  replicas: 5
  selector: { matchLabels: { app: order-service, version: green } }
  template:
    metadata: { labels: { app: order-service, version: green } }
    spec: { containers: [{ name: order-service, image:
 ↪  registry.acme.com/order-service:1.5.0 }] }
---
apiVersion: v1
kind: Service
metadata: { name: order-service }
spec:
  selector: { app: order-service, version: blue }   # ← flip to "green" to cut
 ↪  traffic over
  ports: [{ port: 80, targetPort: 8080 }]
```

```
# Cutover once Green passes smoke tests:
kubectl patch service order-service -p
 ↪  '{"spec":{"selector":{"version":"green"}}}'
# Instant rollback if something's wrong:
kubectl patch service order-service -p
 ↪  '{"spec":{"selector":{"version":"blue"}}}'
```

Canary release with Argo Rollouts — progressive traffic shifting with automated analysis

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: order-service }
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10        # 10% of traffic to the new version
        - pause: { duration: 5m }
        - analysis:
            templates: [{ templateName: error-rate-check }]
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: { name: error-rate-check }
spec:
  metrics:
```

```
    - name: error-rate
      interval: 1m
      failureCondition: result[0] > 0.05     # auto-rollback if >5% error rate
      provider:
        prometheus:
          address: http://prometheus:9090
          query: |

↪   sum(rate(http_server_requests_seconds_count{status=~"5..",app="order-
↪   service"}[1m]))
            /
↪   sum(rate(http_server_requests_seconds_count{app="order-service"}[1m]))
```

Structured logging (JSON, correlation IDs for tracing a request across services)

```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req,
     ↪  HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String correlationId =
         ↪  Optional.ofNullable(req.getHeader("X-Correlation-Id"))
                .orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", correlationId);
        res.setHeader("X-Correlation-Id", correlationId);
        try { chain.doFilter(req, res); } finally { MDC.clear(); }
    }
}
```

```
<!-- logback-spring.xml: JSON structured logs shipped to Loki/ELK -->
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    <includeMdcKeyName>correlationId</includeMdcKeyName>
  </encoder>
</appender>
```

Prometheus metrics via Micrometer (auto-scraped, feeds the canary analysis above)

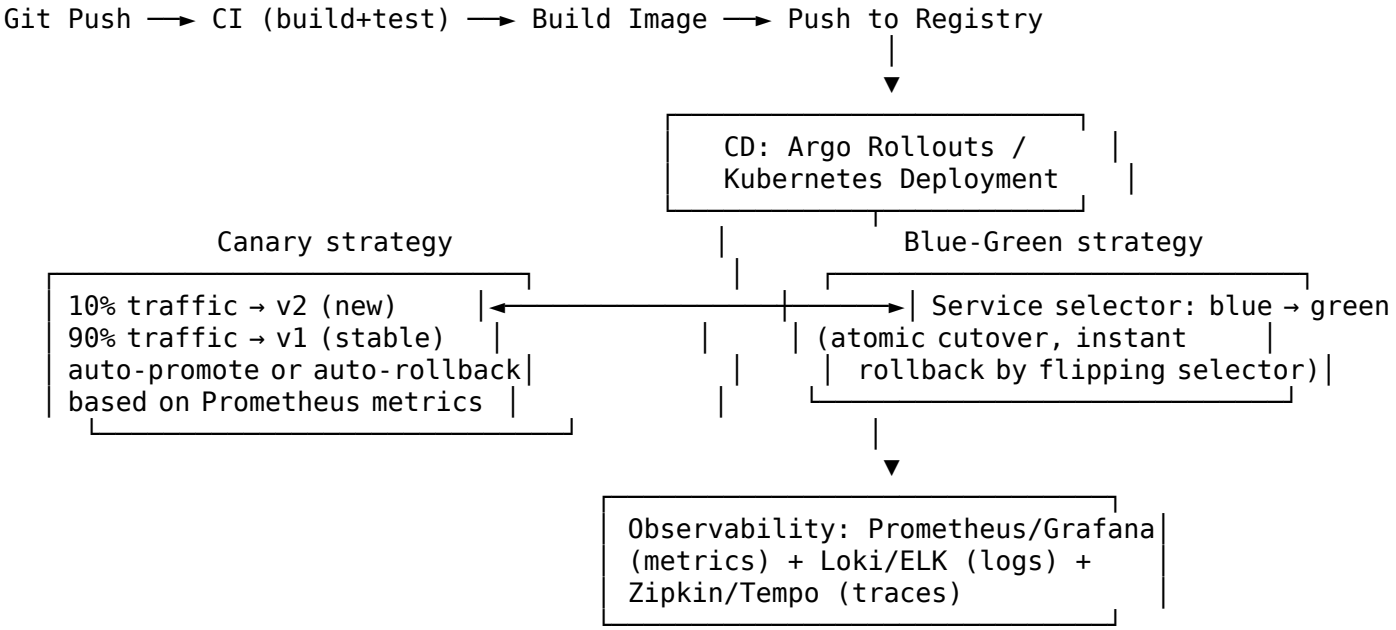
```
management:
  endpoints.web.exposure.include: health, prometheus, metrics
  metrics:
    tags: { application: order-service }
```

```
@Timed(value = "order.create.duration", percentiles = {0.5, 0.95, 0.99})
public OrderResponse placeOrder(OrderRequest req) { ... }
```

4.10.3 Real-Time Usage

- **Google, Netflix, Spotify**: canary releases are the default deployment strategy — every change first goes to 1-5% of production traffic before wider rollout.
- Blue-Green is preferred where **instant, zero-downtime rollback** matters more than gradual exposure (e.g., a database-schema-compatible hotfix).
- Prometheus + Grafana + Loki/ELK is the near-universal open-source observability stack backing these automated release decisions.

4.10.4 Architectural Flow Diagram



4.11 Microservices Resilience4j — Retry, Rate Limiting (Algorithms), Bulkhead

4.11.1 Definition

Resilience4j is a lightweight Java fault-tolerance library providing composable decorators: **Circuit Breaker** (stop calling a failing dependency for a cooldown period), **Retry** (re-attempt transient failures with backoff), **Rate Limiter** (cap calls per time window — algorithms include **Token Bucket**, **Leaky Bucket**, **Fixed Window**, **Sliding Window**), and **Bulkhead** (isolate resources per dependency — e.g., separate thread pools — so one slow dependency can't exhaust threads needed by others, named after ship compartments that stop one flooded section from sinking the whole ship).

4.11.2 Hands-On: Runnable Mini-Project

Retry with exponential backoff

```
@Retry(name = "inventoryService", fallbackMethod = "fallbackCheckStock")
public StockResponse checkStock(Long productId) {
    return inventoryClient.checkStock(productId);
}

public StockResponse fallbackCheckStock(Long productId, Throwable t) {
    return new StockResponse(productId, false);   // conservative fallback
}
```

```
resilience4j:
  retry:
    instances:
      inventoryService:
        max-attempts: 4
        wait-duration: 200ms
        exponential-backoff-multiplier: 2   # 200ms, 400ms, 800ms, 1600ms
        retry-exceptions:
          - java.net.SocketTimeoutException
          - org.springframework.web.client.ResourceAccessException
```

Circuit Breaker (stops calling a service that's clearly down)

```
resilience4j:
  circuitbreaker:
    instances:
      inventoryService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 20
        failure-rate-threshold: 50          # open circuit if >50% of last 20
 ↪  calls fail
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 5
```

```
@CircuitBreaker(name = "inventoryService", fallbackMethod =
 ↪  "fallbackCheckStock")
@Retry(name = "inventoryService")
public StockResponse checkStock(Long productId) { return
 ↪  inventoryClient.checkStock(productId); }
```

Rate Limiter — comparing algorithms

```
// 1. TOKEN BUCKET (Resilience4j's built-in RateLimiter): bucket refills at a
 ↪  fixed rate,
//    each call consumes a token; allows short bursts up to bucket capacity.
@RateLimiter(name = "publicApi")
public CatalogResponse browseCatalog() { return catalogClient.list(); }
```

```
resilience4j:
  ratelimiter:
    instances:
      publicApi:
        limit-for-period: 100      # 100 tokens
        limit-refresh-period: 1s   # refilled every second
        timeout-duration: 0s       # reject immediately if no tokens (no
 ↪  queueing)
```

```
// 2. SLIDING WINDOW (Redis-backed, gateway-level): more precise than
 ↪  fixed-window,
//    avoids the "double burst at window boundary" problem of fixed windows.
public boolean allowRequest(String apiKey) {
    long now = System.currentTimeMillis();
    long windowStart = now - 1000; // 1-second sliding window
    redis.opsForZSet().removeRangeByScore(apiKey, 0, windowStart);
    Long count = redis.opsForZSet().zCard(apiKey);
    if (count != null && count >= 100) return false;   // limit: 100 req/sec
    redis.opsForZSet().add(apiKey, UUID.randomUUID().toString(), now);
    return true;
}

// 3. LEAKY BUCKET (smooths bursts into a constant outflow rate — good for
 ↪  protecting
//    a fixed-capacity downstream like a legacy mainframe):
public class LeakyBucketLimiter {
    private final Queue<Runnable> queue = new ConcurrentLinkedQueue<>();
    // A fixed-rate scheduler drains ("leaks") one request from the queue
     ↪  every N ms,
    // regardless of how fast requests arrive — smooths bursts into steady
     ↪  throughput.
}
```

Bulkhead — isolate thread pools so Payment's slowness can't starve Inventory calls

```
resilience4j:
  thread-pool-bulkhead:
    instances:
      paymentService:
        max-thread-pool-size: 10
        core-thread-pool-size: 5
        queue-capacity: 20
      inventoryService:
        max-thread-pool-size: 15
        core-thread-pool-size: 8
        queue-capacity: 30
```

```java
@Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
public CompletableFuture<PaymentResponse> charge(PaymentRequest req) {
    return CompletableFuture.supplyAsync(() -> paymentClient.charge(req));
}
```

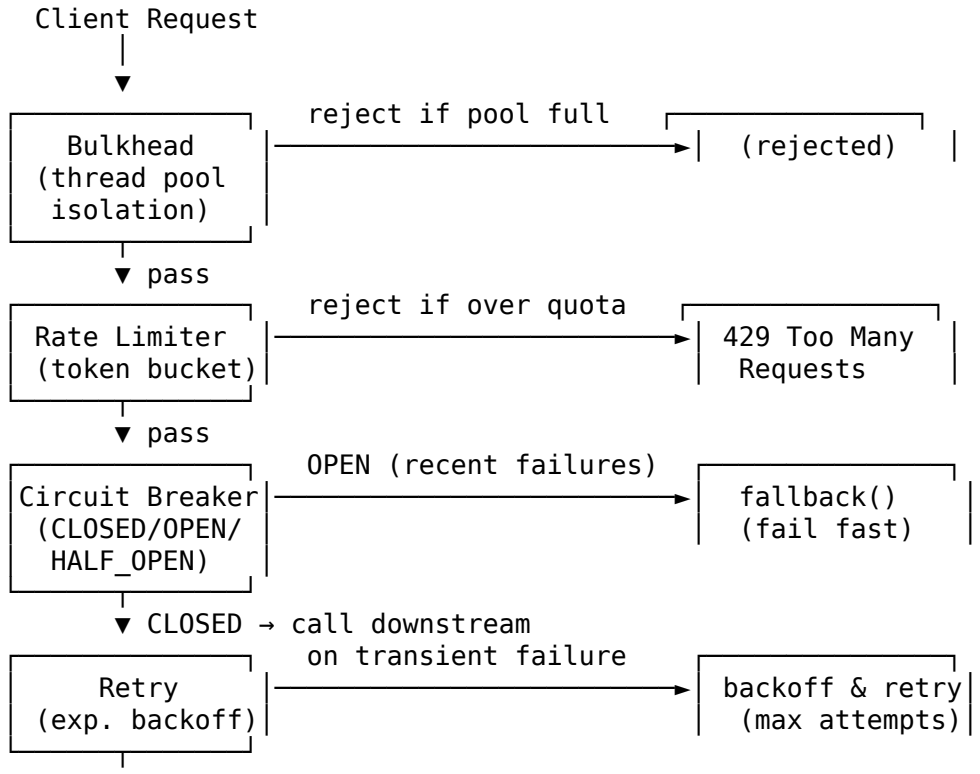
Composing all four on one call (the recommended order: Bulkhead → RateLimiter → CircuitBreaker → Retry)

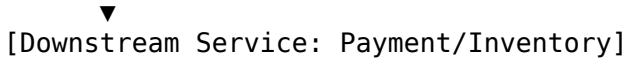
```java
@Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
@RateLimiter(name = "paymentService")
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackCharge")
@Retry(name = "paymentService")
public CompletableFuture<PaymentResponse> charge(PaymentRequest req) {
    return CompletableFuture.supplyAsync(() -> paymentClient.charge(req));
}
```

4.11.3 Real-Time Usage

- **Netflix's Hystrix** (Resilience4j's spiritual predecessor) pioneered circuit breakers + bulkheads to survive its own dependency failures at massive scale; Resilience4j is the modern, actively-maintained successor.
- Payment gateways commonly use **token-bucket** rate limiting to allow occasional bursts (checkout spikes) while enforcing a steady long-term cap per merchant.
- Bulkheading Payment vs. Inventory thread pools prevents a slow third-party payment gateway outage from also freezing unrelated inventory checks — a classic **cascading failure** prevention technique.

4.11.4 Architectural Flow Diagram




[Downstream Service: Payment/Inventory]

5 Part IV: Migration Playbook — Monolith to Microservices

Everything so far has shown the *end states* (monolith, modular monolith, microservices). This part is the **journey between them** — a battle-tested, six-phase migration playbook (based on Chris Richardson's *Microservices Patterns* migration strategy and Martin Fowler's **Strangler Fig** pattern) for taking a live, running monolith to microservices **without a risky big-bang rewrite** and **without downtime**. Each phase below builds directly on the e-commerce Order Platform used throughout this document.

```
Phase 1        Phase 2        Phase 3          Phase 4         Phase 5          Phase 6
Decouple  ──►  Develop   ──►  Implement   ──►  Deploy MS  ──►  Implement MS ──►  Decommission
Monolith       Local MS       Local MS         to Cloud        in the Cloud      Monolith
(facade +      (identify +    (migrate data +  (API Gateway +  (migrate data +   (unplug
 mediation)     interface +    test parallel +  deploy)         run parallel +    monolith)
                build)         cut traffic)                     cut to gateway)
```

5.1 Phase 1: Decouple the Monolith

5.1.1 Definition

Before any code is extracted, the monolith must be made *extractable*. Two sub-steps: **(i) Add a Service Layer / Facade** — pull business logic out of fat controllers into a clean service layer so there's a single seam to cut along later; **(ii) Add a Service Mediation Layer** — introduce a routing/mediator component in front of that facade whose only job, for now, is to decide *whether a request is served by the old monolith code or a new microservice* — even though today every request still goes to the monolith. This mediator is what lets every later phase shift traffic gradually instead of flipping a single switch.

5.1.2 Hands-On: Runnable Mini-Project

Before — controller talks directly to repositories (tightly coupled, nothing to cut along):

```
@RestController
public class OrderController {
    private final OrderRepository orderRepo;
    private final ProductRepository productRepo;

    @PostMapping("/orders")
    public OrderResponse create(@RequestBody OrderRequest req) {
        Product product = productRepo.findById(req.productId()).orElseThrow();
        Order order = orderRepo.save(new Order(req.productId(), req.qty(),
↪   OrderStatus.CREATED));
        return new OrderResponse(order.getId(), "CREATED");
    }
}
```

Step (i) — Add a Service Layer / Facade: move all business logic behind one interface

```
public interface OrderFacade {
    OrderResponse createOrder(OrderRequest req);
}

@Service
public class LegacyOrderFacade implements OrderFacade {
```

```
    private final OrderRepository orderRepo;
    private final ProductRepository productRepo;

    @Override
    public OrderResponse createOrder(OrderRequest req) {
        Product product = productRepo.findById(req.productId()).orElseThrow();
        Order order = orderRepo.save(new Order(req.productId(), req.qty(),
 ↪  OrderStatus.CREATED));
        return new OrderResponse(order.getId(), "CREATED");
    }
}

// Controller is now a thin adapter — the only thing that changed from the
 ↪  caller's point of view: nothing.
@RestController
public class OrderController {
    private final OrderFacade orderFacade;
    public OrderController(OrderFacade orderFacade) { this.orderFacade =
     ↪  orderFacade; }

    @PostMapping("/orders")
    public OrderResponse create(@RequestBody OrderRequest req) {
        return orderFacade.createOrder(req);
    }
}
```

Step (ii) — Add a Service Mediation Layer: a router that will later decide legacy vs. new microservice

```
@Component
public class OrderServiceMediator implements OrderFacade {

    @Value("${migration.order-service.enabled:false}")   // OFF for all of
     ↪  Phase 1
    private boolean useNewOrderService;

    private final OrderFacade legacyOrderFacade;          // existing monolith
     ↪  code path
    private final ObjectProvider<OrderFacade> newOrderServiceClient; // not
     ↪  implemented yet — Phase 2/3

    public OrderServiceMediator(@Qualifier("legacyOrderFacade") OrderFacade
     ↪  legacyOrderFacade,
                                ObjectProvider<OrderFacade>
 ↪  newOrderServiceClient) {
        this.legacyOrderFacade = legacyOrderFacade;
        this.newOrderServiceClient = newOrderServiceClient;
    }

    @Override
    public OrderResponse createOrder(OrderRequest req) {
        if (useNewOrderService && newOrderServiceClient.getIfAvailable() !=
         ↪  null) {
            return newOrderServiceClient.getObject().createOrder(req);
```

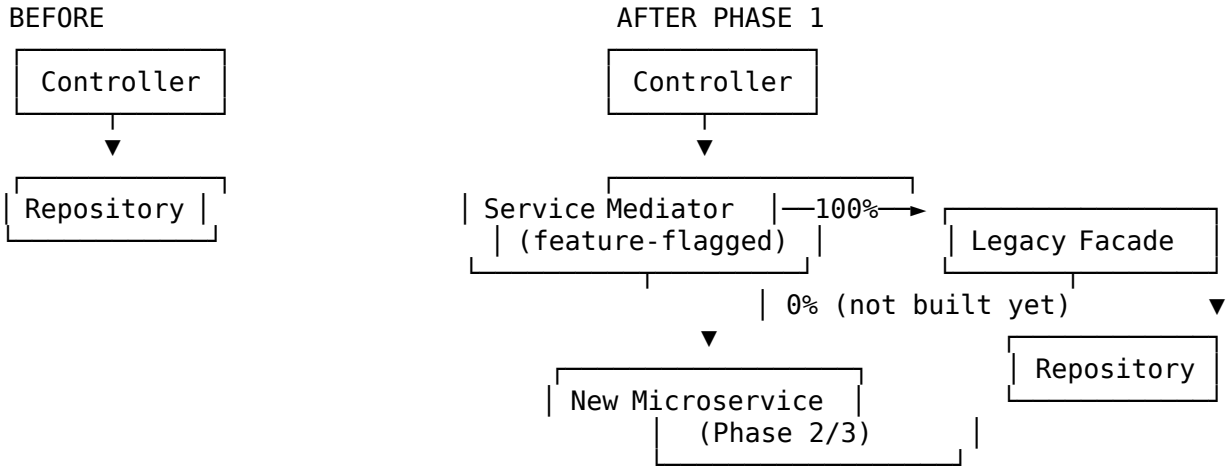
```
        }
        return legacyOrderFacade.createOrder(req);   // 100% of traffic today
    }
}
```

```
# application.yml — the single switch every later phase will flip, service by service
migration:
  order-service:
    enabled: false   # Phase 1: stays false. Flips per-service as each one goes live.
```

5.1.3 Real-Time Usage

- This is exactly how **Shopify** and **eBay** began their multi-year migrations off large Rails/Java monoliths — introducing a facade and a routing seam *before* writing a single line of the new service.
- The mediator pattern here is a miniature, code-level version of the API Gateway routing you'll build formally in Phase 4 — introducing it early means the eventual cutover is a config change, not a rewrite.

5.1.4 Architectural Flow Diagram



5.2 Phase 2: Develop Local Microservices

5.2.1 Definition

With a seam in place, the next phase builds the new microservice **without touching production traffic yet**: **(i) Identify the microservice** using the decomposition techniques from Part II (bounded context, business capability); **(ii) Develop interface definitions** — write the contract (OpenAPI/gRPC proto) *before* the implementation, so the mediator and any future consumers can be coded against a stable API; **(iii) Develop the microservice** itself, initially reading from the **same shared database** the monolith already uses (data migration happens in Phase 3) — the goal here is purely to prove the service's business logic works, in isolation, "locally."

5.2.2 Hands-On: Runnable Mini-Project

Step (i) — Identify the microservice via bounded context (reusing Part II's decomposition):

```
Bounded Context chosen: Ordering
Owns: Order, OrderLine
Talks to: Catalog (read-only, for price/stock at order time)
```

Step (ii) — Develop the interface definition FIRST (contract-first, OpenAPI)

```
# order-service-api.yaml
openapi: 3.0.3
info: { title: Order Service API, version: 1.0.0 }
paths:
  /orders:
    post:
      operationId: createOrder
      requestBody:
        content:
          application/json:
            schema: { $ref: '#/components/schemas/OrderRequest' }
      responses:
        '201':
          content:
            application/json:
              schema: { $ref: '#/components/schemas/OrderResponse' }
components:
  schemas:
    OrderRequest:  { type: object, properties: { productId: {type: integer},
 ↪  qty: {type: integer} } }
    OrderResponse: { type: object, properties: { orderId: {type: integer},
 ↪  status: {type: string} } }
```

```
<!-- pom.xml: generate the Java interface + DTOs directly from the YAML
 ↪  contract -->
<plugin>
  <groupId>org.openapitools</groupId>
  <artifactId>openapi-generator-maven-plugin</artifactId>
  <configuration>
    <inputSpec>${project.basedir}/order-service-api.yaml</inputSpec>
    <generatorName>spring</generatorName>
    <apiPackage>com.acme.order.api</apiPackage>
```

```
  </configuration>
</plugin>
```

Step (iii) — Develop the microservice, implementing the generated interface, still on the shared DB

```
@RestController
public class OrderServiceController implements OrdersApi {   // generated
 ↪  interface from the contract

    private final OrderRepository orderRepo;        // points at the SAME
     ↪  shared DB as the monolith, for now
    private final ProductClient productClient;       // reads Catalog data —
     ↪  still via direct DB read at this stage

    @Override
    public ResponseEntity<OrderResponse> createOrder(OrderRequest req) {
        Product product = productClient.findById(req.getProductId());
        if (product.getStock() < req.getQty()) {
            return ResponseEntity.badRequest().build();
        }
        Order order = orderRepo.save(new Order(req.getProductId(),
 ↪  req.getQty(), OrderStatus.CREATED));
        return ResponseEntity.status(201).body(new
         ↪  OrderResponse(order.getId(), "CREATED"));
    }
}
```

```
# order-service/application.yml — new service, but same physical database as
 ↪  the monolith (Phase 2 only)
spring:
  application: { name: order-service }
  datasource:
    url: jdbc:postgresql://shared-monolith-db:5432/shop   # NOT its own DB yet
 ↪  — that's Phase 3
```

Wiring the mediator to the new service, still behind the flag (off in production, on in a test environment)

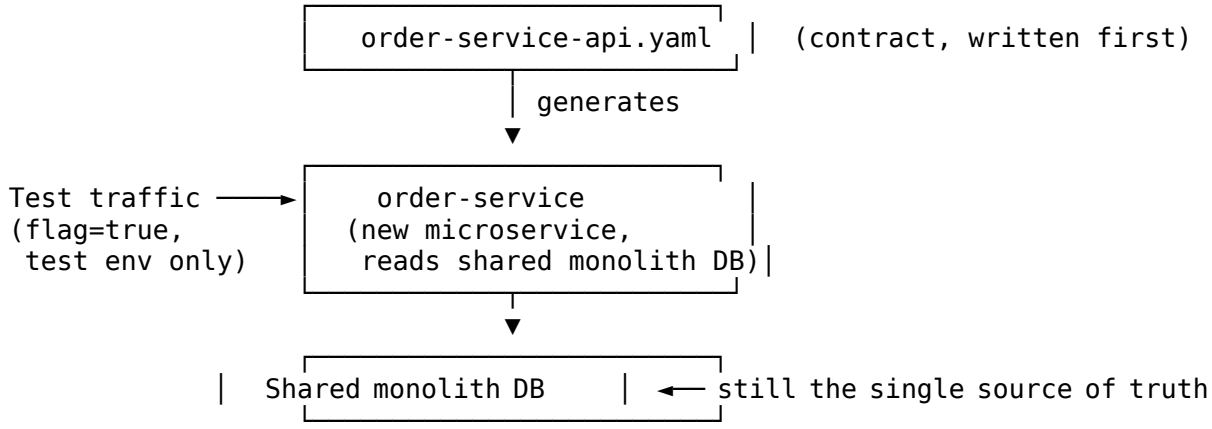
```
@Component
@Qualifier("newOrderService")
public class NewOrderServiceClient implements OrderFacade {
    private final RestClient restClient;   // calls order-service over HTTP,
     ↪  using the generated client

    @Override
    public OrderResponse createOrder(OrderRequest req) {
        return restClient.post().uri("http://order-service/orders")
                .body(req).retrieve().body(OrderResponse.class);
    }
}
```

5.2.3 Real-Time Usage

- **Contract-first development** (OpenAPI/proto before code) is standard at any org running dozens of teams in parallel — it lets the API Gateway, the mediator, and consumer teams all build against a stable contract while the service internals are still being written.
- Building against the **shared database first** is a deliberate, common step — it isolates "does the business logic work" from "does the data migration work," so you debug one variable at a time.

5.2.4 Architectural Flow Diagram



5.3 Phase 3: Implement Local Microservices

5.3.1 Definition

The new microservice works against the shared DB — now it must **stand on its own data** and **prove itself against real traffic before being trusted with it**: **(i) Migrate data** — give the service its own database and keep it in sync with the monolith's during the transition (dual-write or CDC); **(ii) Testing / parallel run** — send real production requests to *both* the legacy path and the new service, compare the results, but only return the legacy response to the user ("shadow traffic"); **(iii) Sewing channels to microservices** — once shadow results match, start actually cutting a small, real percentage of traffic over to the new service through the mediator.

5.3.2 Hands-On: Runnable Mini-Project

Step (i) — Migrate data: Debezium CDC streams monolith DB changes into the new service's own database

```
// debezium-connector-config.json — captures changes from the monolith's
 ↪  "orders" table
{
  "name": "orders-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "shared-monolith-db",
    "table.include.list": "public.orders",
    "topic.prefix": "monolith-cdc"
  }
}
```

```
// order-service consumes the CDC stream and populates its OWN database in
 ↪  real time
@KafkaListener(topics = "monolith-cdc.public.orders", groupId =
 ↪  "order-service-sync")
public void onMonolithOrderChange(DebeziumChangeEvent event) {
    Order order = mapToOrder(event.getAfter());
    ownOrderRepository.save(order);   // now order-service has its own,
 ↪  independently-owned copy
}
```

Step (ii) — Testing / Parallel Run: shadow traffic through the mediator (compare, don't cut over yet)

```
@Component
public class ShadowingOrderMediator implements OrderFacade {
    private final OrderFacade legacyOrderFacade;
    private final OrderFacade newOrderServiceClient;
    private final ExecutorService shadowExecutor =
     ↪  Executors.newFixedThreadPool(4);
    private static final Logger diffLog =
     ↪  LoggerFactory.getLogger("shadow-diff");

    @Override
    public OrderResponse createOrder(OrderRequest req) {
        OrderResponse legacyResponse = legacyOrderFacade.createOrder(req);  //
 ↪  authoritative — returned to user
```

```
        shadowExecutor.submit(() -> {                                    //
↪   fire-and-forget, never blocks the user
            try {
                OrderResponse shadowResponse =
↪   newOrderServiceClient.createOrder(req);
                if (!legacyResponse.status().equals(shadowResponse.status())) {
                    diffLog.warn("MISMATCH legacy={} new={} request={}",
                                 legacyResponse, shadowResponse, req);
                }
            } catch (Exception e) {
                diffLog.warn("Shadow call failed: {}", e.getMessage());
            }
        });
        return legacyResponse;
    }
}
```

Step (iii) — Sewing channels: start cutting a real, small % of live traffic to the microservice

```
@Component
public class PercentageCutoverMediator implements OrderFacade {
    @Value("${migration.order-service.traffic-percent:0}")
    private int trafficPercent;      // e.g. 5, then 25, then 100 — raised
     ↪  gradually

    private final OrderFacade legacyOrderFacade;
    private final OrderFacade newOrderServiceClient;

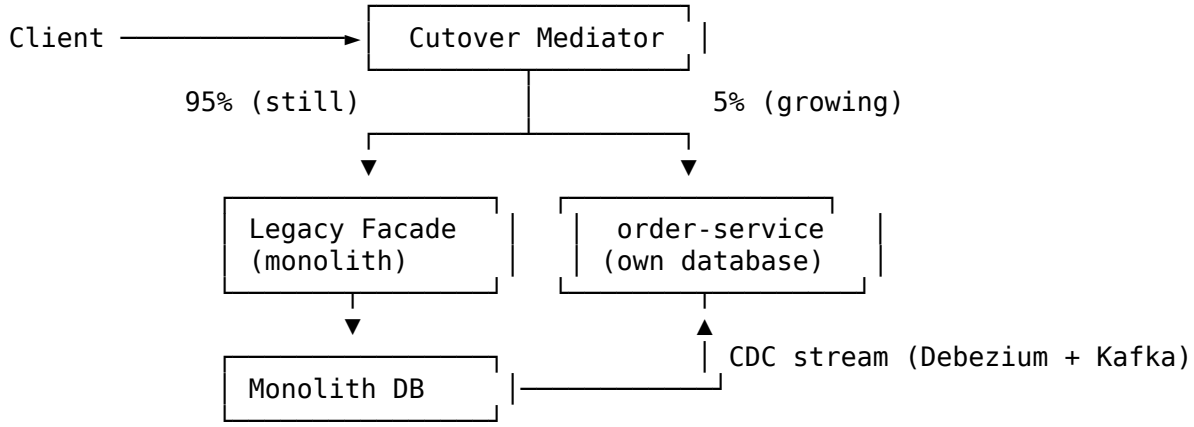
    @Override
    public OrderResponse createOrder(OrderRequest req) {
        boolean routeToNew = ThreadLocalRandom.current().nextInt(100) <
         ↪  trafficPercent;
        return routeToNew ? newOrderServiceClient.createOrder(req) :
         ↪  legacyOrderFacade.createOrder(req);
    }
}
```

```
migration:
  order-service:
    traffic-percent: 5   # raise to 25, 50, 100 over successive days as
↪   confidence builds
```

5.3.3 Real-Time Usage

- **Shadow traffic / "dark launching"** is exactly how **GitHub** and **Stripe** validate rewritten payment and API-serving code paths — comparing outputs silently for days/weeks before a single real user is exposed to the new path.
- CDC-based data sync (Debezium) avoids the classic "dual write" bug — the monolith keeps writing to its own DB unaware anything has changed, while the new service listens and stays eventually consistent.

5.3.4 Architectural Flow Diagram



5.4 Phase 4: Deploy Microservices to the Cloud

5.4.1 Definition

The microservice has proven itself running "locally" (on-premises, alongside the monolith). Now it graduates to being a real, independently-deployed system: **(i) Implement an API Gateway** — replace the in-process mediator with a proper edge gateway that is the new system boundary for external clients; **(ii) Deploy the microservice** — containerize it and deploy to Kubernetes/the cloud, independent of the monolith's release cycle for the first time.

5.4.2 Hands-On: Runnable Mini-Project

Step (i) — Implement the API Gateway, replacing the in-process mediator with a network-level router

```
# Spring Cloud Gateway takes over the job the OrderServiceMediator used to do
 ↪  in-process
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service              # now a real, independently
 ↪  deployed service
          predicates: [ "Path=/api/orders/**" ]
          filters:
            - name: Retry
              args: { retries: 2, statuses: BAD_GATEWAY }
        - id: legacy-monolith-route
          uri: lb://legacy-monolith             # everything not yet migrated
 ↪  still goes here
          predicates: [ "Path=/**" ]
```

```
// The gateway now owns the traffic-percent decision that used to live in Java
 ↪  code
@Component
public class WeightedRoutePredicateFactory extends
 ↪  AbstractRoutePredicateFactory<WeightedRoutePredicateFactory.Config> {
    // Spring Cloud Gateway's built-in Weight filter does exactly this:
    // spring.cloud.gateway.routes[].metadata.weight: { group: "order",
     ↪  weight: 5 }  → 5% to order-service
}
```

Step (ii) — Deploy the microservice to Kubernetes, independent of the monolith

```
FROM eclipse-temurin:17-jre-alpine
COPY target/order-service.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service }
spec:
  replicas: 3
  selector: { matchLabels: { app: order-service } }
```

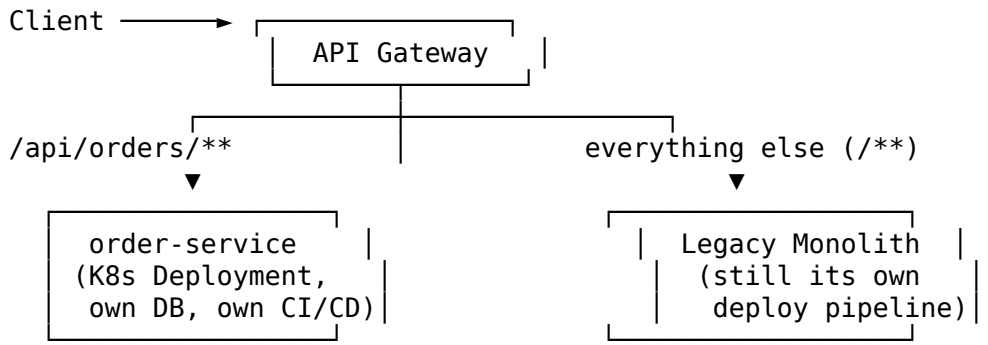
```
  template:
    metadata: { labels: { app: order-service } }
    spec:
      containers:
        - name: order-service
          image: registry.acme.com/order-service:1.0.0
          ports: [{ containerPort: 8080 }]
          readinessProbe: { httpGet: { path: /actuator/health/readiness, port:
 ↪  8080 } }
---
apiVersion: v1
kind: Service
metadata: { name: order-service }
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8080 }]
```

```
# The monolith keeps deploying on its own old schedule/pipeline — completely
 ↪  decoupled now
kubectl apply -f order-service-deployment.yaml
kubectl rollout status deployment/order-service
```

5.4.3 Real-Time Usage

- This is the point at which a migrating service starts appearing in the **same reference architecture diagrams as Part III** — behind a real API Gateway, in its own Kubernetes Deployment, with its own CI/CD pipeline (Part III's Canary/Blue-Green strategies now apply to it directly).
- Companies typically keep the **legacy catch-all route** (Path=/** → monolith) in the gateway for months or years, migrating one path at a time — this is the Strangler Fig in its purest form, visible directly in gateway config.

5.4.4 Architectural Flow Diagram



5.5 Phase 5: Implement Microservices in the Cloud

5.5.1 Definition

The service is deployed to the cloud, but it may still be reading from an on-premises database, and the gateway may still be sending it only a fraction of traffic. This phase finishes the job: **(i) Migrate data to the cloud** — move the service's database itself to a managed cloud data store; **(ii) Run parallel in the cloud** — validate the cloud deployment against the on-prem one exactly as Phase 3 validated the microservice against the monolith; **(iii) Sew the channels to the API Gateway** — ramp the gateway's routing weight from a small percentage up to 100%, using the same Canary/Blue-Green machinery from Part III.

5.5.2 Hands-On: Runnable Mini-Project

Step (i) — Migrate the service's database to a managed cloud instance

```
# AWS DMS (Database Migration Service) — continuous replication, on-prem
 ↪  Postgres -> RDS
aws dms create-replication-task \
  --replication-task-identifier order-db-to-cloud \
  --migration-type full-load-and-cdc \
  --source-endpoint-arn arn:aws:dms:...:endpoint:onprem-order-db \
  --target-endpoint-arn arn:aws:dms:...:endpoint:rds-order-db
```

```
# order-service switches its datasource once replication lag is ~0
spring:
  datasource:
    url: jdbc:postgresql://order-db.cluster-xyz.rds.amazonaws.com:5432/orders
```

Step (ii) — Run parallel in the cloud: validate cloud deployment against on-prem before trusting it

```
@Component
public class CloudParityValidator {
    private final OrderFacade onPremOrderService;
    private final OrderFacade cloudOrderService;
    private static final Logger parityLog =
     ↪  LoggerFactory.getLogger("cloud-parity");

    @Scheduled(fixedDelay = 60000)
    public void validateSample() {
        OrderRequest sample = sampleGenerator.generate();
        OrderResponse onPrem = onPremOrderService.createOrder(sample);
        OrderResponse cloud = cloudOrderService.createOrder(sample);
        if (!onPrem.status().equals(cloud.status())) {
            parityLog.warn("Cloud parity mismatch: onPrem={} cloud={}",
 ↪  onPrem, cloud);
        }
    }
}
```

Step (iii) — Sew the channels to the API Gateway: ramp traffic weight using Canary (reusing Part III's Argo Rollouts)

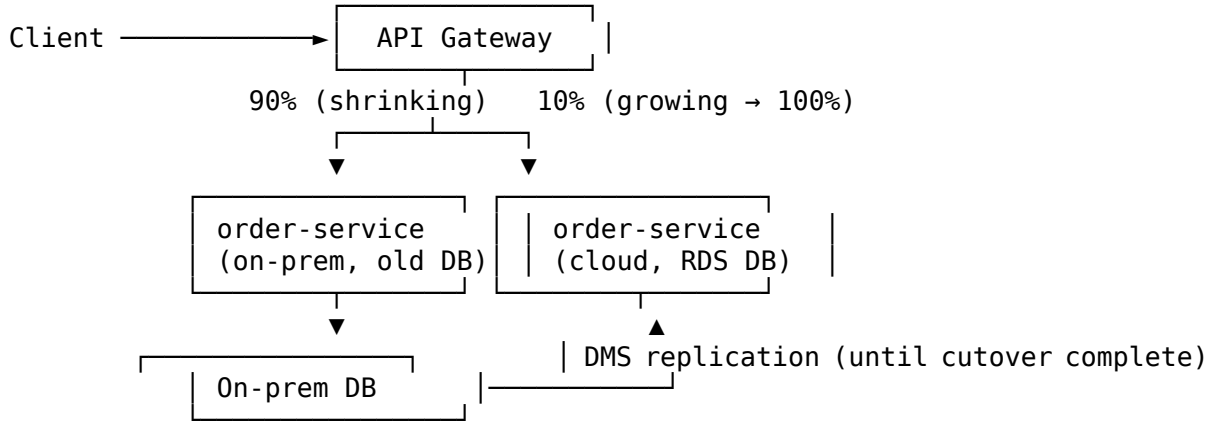
```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
```

```yaml
metadata: { name: order-service-cloud-cutover }
spec:
  strategy:
    canary:
      steps:
        - setWeight: 10     # 10% of gateway traffic now hits the CLOUD
↪  deployment
        - pause: { duration: 1h }
        - analysis: { templates: [{ templateName: error-rate-check }] }   #
↪  from Part III
        - setWeight: 50
        - pause: { duration: 4h }
        - setWeight: 100    # cloud is now fully live; on-prem instance can be
↪  drained
```

5.5.3 Real-Time Usage

- This mirrors real cloud migrations at **Capital One** and **Netflix** (both well-documented): data replication first, parallel validation second, and only then a weighted, reversible traffic cutover at the gateway — never a single flag flip for a production database.
- Reusing the exact Canary machinery from Part III here is deliberate — once a service is on the platform, *every* future change (including "where it physically runs") goes through the same safe-release pipeline.

5.5.4 Architectural Flow Diagram



5.6 Phase 6: Decommission the Monolith

5.6.1 Definition

Once every bounded context has run the full Phase 1-5 cycle and the API Gateway is routing 100% of traffic to microservices, the monolith becomes dead code sitting behind a route nobody uses: **(i) Unplug the monolith** — remove its routes from the gateway, decommission its compute and database, and delete the now-unused mediator/facade scaffolding from Phase 1.

5.6.2 Hands-On: Runnable Mini-Project

Step (i) — Unplug the monolith: remove its gateway route once every path has a microservice owner

```yaml
# BEFORE (Phase 4/5): legacy catch-all still present
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates: [ "Path=/api/orders/**" ]
        - id: catalog-service-route
          uri: lb://catalog-service
          predicates: [ "Path=/api/catalog/**" ]
        - id: legacy-monolith-route          # ← delete this once nothing needs
 ↪  it
          uri: lb://legacy-monolith
          predicates: [ "Path=/**" ]

# AFTER (Phase 6): the monolith route is gone — every path is owned by a
 ↪  microservice
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates: [ "Path=/api/orders/**" ]
        - id: catalog-service-route
          uri: lb://catalog-service
          predicates: [ "Path=/api/catalog/**" ]
```

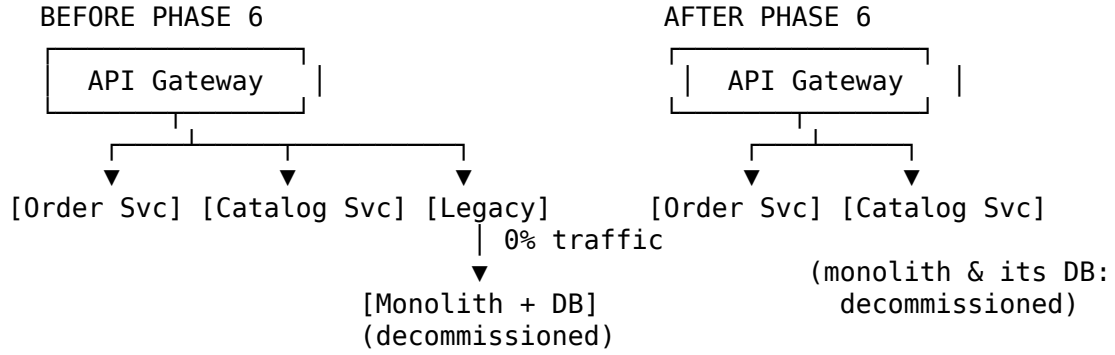
```bash
# Decommission compute and data safely, in order — never delete data first
kubectl scale deployment/legacy-monolith --replicas=0        # 1. stop serving
 ↪  traffic
# ...wait one full release cycle with monitoring, confirm zero requests reach
 ↪  it...
kubectl delete deployment/legacy-monolith                    # 2. remove compute
pg_dump legacy-monolith-db > final-archive-$(date +%F).sql    # 3. take a
 ↪  final archival backup
aws rds delete-db-instance --db-instance-identifier legacy-monolith-db
 ↪  --skip-final-snapshot false  # 4. decommission DB
```

```
// Clean up the Phase-1 scaffolding — the mediator and legacy facade have no
 ↪  purpose left
// DELETE: OrderServiceMediator, PercentageCutoverMediator, LegacyOrderFacade,
 ↪  migration.* config keys
```

5.6.3 Real-Time Usage

- Large migrations (e.g. **Amazon's** original monolith-to-services move) took *years*, decommissioning one bounded context at a time — Phase 6 for the "Catalog" context might happen while "Ordering" is still in Phase 3.
- Teams intentionally keep a **final archival backup** of the monolith's database even after deletion — a cheap insurance policy against a bounded context that was decomposed incorrectly.

5.6.4 Architectural Flow Diagram



5.7 Migration Playbook — Summary

Phase	Goal	Key Artifact
1. Decouple Monolith	Create a seam to cut along	Service Facade + Mediator
2. Develop Local Microservices	Prove business logic in isolation	Contract-first microservice on shared DB
3. Implement Local Microservices	Prove correctness against real traffic	Own database (CDC-synced) + shadow traffic
4. Deploy Microservices to Cloud	Make it independently deployable	API Gateway route + K8s Deployment
5. Implement MS in the Cloud	Make the cloud copy the source of truth	Cloud DB migration + Canary cutover
6. Decommission Monolith	Remove what's no longer needed	Deleted routes, deleted compute/DB

Repeat Phases 1-6 **per bounded context** — Catalog, Ordering, Payments, and Inventory each run this cycle on their own timeline, which is exactly why the Strangler Fig approach scales to monoliths far too large to rewrite in one push.

6 Conclusion

This guide walked the full arc: **Monolith → Modular Monolith → Microservices/Event-Driven → Serverless**, the cross-cutting concerns that make microservices production-grade — **decomposition, communication, data management, caching, deployment, resilience, and safe releases with observability** — and finally the **six-phase migration playbook** for taking a live monolith to microservices safely, one bounded context at a time. Each pattern includes a runnable Spring Boot + Kafka + PostgreSQL/Cassandra mini-project so you can stand up the concept locally with `docker compose up` and see the architecture in action end-to-end.